

字符串

Scintilla

Tsinghua University

2026 年 5 月 4 日

1 记号

2 border 相关

3 ACAM

4 回文串相关

5 Z 函数

6 后缀数据结构

7 基本子串结构

8 References

记号

- 设字符集为 Σ 。若无特殊说明，一般认为 Σ 为 26 个小写英文字母。

记号

- 设字符集为 Σ 。若无特殊说明，一般认为 Σ 为 26 个小写英文字母。
- 对于字符串 s ，设 $|s|$ 表示其长度。本文中若无特殊说明，一般用 n 代指 $|s|$ ，用 m 指代 $|t|$ ，并且默认 n, m 同阶。

记号

- 设字符集为 Σ 。若无特殊说明，一般认为 Σ 为 26 个小写英文字母。
- 对于字符串 s ，设 $|s|$ 表示其长度。本文中若无特殊说明，一般用 n 代指 $|s|$ ，用 m 指代 $|t|$ ，并且默认 n, m 同阶。
- 对于字符串 s ， $s[i]$ 表示 s 的第 i 个字符， $s[i..j]$ 表示字符串 $s[i]s[i+1]\cdots s[j]$ ，下标从 1 开始。

记号

- 设字符集为 Σ 。若无特殊说明，一般认为 Σ 为 26 个小写英文字母。
- 对于字符串 s ，设 $|s|$ 表示其长度。本文中若无特殊说明，一般用 n 代指 $|s|$ ，用 m 指代 $|t|$ ，并且默认 n, m 同阶。
- 对于字符串 s ， $s[i]$ 表示 s 的第 i 个字符， $s[i..j]$ 表示字符串 $s[i]s[i+1]\cdots s[j]$ ，下标从 1 开始。
- 对于字符串 s ，设 $\text{pre}(s, i)$ 表示字符串 $s[1]s[2]\cdots s[i]$ ，即 s 的长度为 i 的前缀； $\text{suf}(s, i)$ 表示字符串 $s[n-i+1]\cdots s[n-1]s[n]$ ，即 s 的长度为 i 的后缀。

记号

- 设字符集为 Σ 。若无特殊说明，一般认为 Σ 为 26 个小写英文字母。
- 对于字符串 s ，设 $|s|$ 表示其长度。本文中若无特殊说明，一般用 n 代指 $|s|$ ，用 m 指代 $|t|$ ，并且默认 n, m 同阶。
- 对于字符串 s ， $s[i]$ 表示 s 的第 i 个字符， $s[i..j]$ 表示字符串 $s[i]s[i+1]\cdots s[j]$ ，下标从 1 开始。
- 对于字符串 s ，设 $\text{pre}(s, i)$ 表示字符串 $s[1]s[2]\cdots s[i]$ ，即 s 的长度为 i 的前缀； $\text{suf}(s, i)$ 表示字符串 $s[n-i+1]\cdots s[n-1]s[n]$ ，即 s 的长度为 i 的后缀。
- 对于字符串 s, t ，设 st 表示其拼接，即 $st = s[1]s[2]\cdots s[n]t[1]t[2]\cdots t[m]$ 。定义 $s^1 = s, s^i = s^{i-1}s, s^R = s[n]s[n-1]\cdots s[1]$ 。

- 1 记号
- 2 border 相关
- 3 ACAM
- 4 回文串相关
- 5 Z 函数
- 6 后缀数据结构
- 7 基本子串结构
- 8 References

定义

对于 $p \in [1, n]$, 称 p 是 s 的 period, 当且仅当 $\forall i \in [1, n - p], s[i] = s[i + p]$ 。

定义

对于 $p \in [1, n]$, 称 p 是 s 的 period, 当且仅当 $\forall i \in [1, n - p], s[i] = s[i + p]$ 。

对于 $p \in [1, n)$, 称 $\text{pre}(s, p)$ 是 s 的 border, 当且仅当 $\text{pre}(s, p) = \text{suf}(s, p)$ 。

定义

对于 $p \in [1, n]$, 称 p 是 s 的 period, 当且仅当 $\forall i \in [1, n - p]$, $s[i] = s[i + p]$ 。

对于 $p \in [1, n)$, 称 $\text{pre}(s, p)$ 是 s 的 border, 当且仅当 $\text{pre}(s, p) = \text{suf}(s, p)$ 。

不难发现, $\text{pre}(s, p)$ 是 s 的 border 当且仅当 $n - p$ 是 s 的 period。

KMP

KMP 算法能够在 $\mathcal{O}(n)$ 的时间内求出 s 的所有 border。

KMP

KMP 算法能够在 $\mathcal{O}(n)$ 的时间内求出 s 的所有 border。

具体地，KMP 算法求出了一个长度为 n 的数组 fail ， fail_i 表示 $\text{pre}(s, i)$ 的最长 border 长度。

KMP

KMP 算法能够在 $\mathcal{O}(n)$ 的时间内求出 s 的所有 border。

具体地，KMP 算法求出了一个长度为 n 的数组 fail ， fail_i 表示 $\text{pre}(s, i)$ 的最长 border 长度。

这个数组通过增量法求出。算法维护一个指针 j ，表示当前字符串的最长 border 长度。每次新加入一个字符时尝试能否继续匹配，若不能则令 $j \leftarrow \text{fail}[j]$ ，直至能够匹配或 $j = 0$ 。

KMP

KMP 算法能够在 $\mathcal{O}(n)$ 的时间内求出 s 的所有 border。

具体地，KMP 算法求出了一个长度为 n 的数组 fail ， fail_i 表示 $\text{pre}(s, i)$ 的最长 border 长度。

这个数组通过增量法求出。算法维护一个指针 j ，表示当前字符串的最长 border 长度。每次新加入一个字符时尝试能否继续匹配，若不能则令 $j \leftarrow \text{fail}[j]$ ，直至能够匹配或 $j = 0$ 。

设 $\mathbf{Bd}(i)$ 表示 $\text{pre}(s, i)$ 的所有 border。若 $\text{fail}[i] = 0$ ，那么 $\mathbf{Bd}(i) = \emptyset$ ；否则

$$\mathbf{Bd}(i) = \{\text{fail}[i]\} \cup \mathbf{Bd}(\text{fail}[i]).$$

Periodicity Lemma

Theorem 1.1 (Weak Periodicity Lemma)

若 $p + q \leq n$, 且 p 和 q 同为 s 的 period, 那么 $\gcd(p, q)$ 也是 s 的 period。

Periodicity Lemma

Theorem 1.1 (Weak Periodicity Lemma)

若 $p + q \leq n$, 且 p 和 q 同为 s 的 period, 那么 $\gcd(p, q)$ 也是 s 的 period。

Proof

不妨设 $p < q$ 。对于任意 $1 \leq i \leq n - q + p$, 因为 $i - p \geq 1$ 和 $i + q \leq n$ 至少有一个满足, 可以通过 $s[i - p]$ 或 $s[i + q]$ 推出 $s[i] = s[i - p + q]$, 进而 $q - p$ 也是 s 的 period。

Periodicity Lemma

Theorem 1.1 (Weak Periodicity Lemma)

若 $p + q \leq n$, 且 p 和 q 同为 s 的 period, 那么 $\gcd(p, q)$ 也是 s 的 period.

Proof

不妨设 $p < q$. 对于任意 $1 \leq i \leq n - q + p$, 因为 $i - p \geq 1$ 和 $i + q \leq n$ 至少有一个满足, 可以通过 $s[i - p]$ 或 $s[i + q]$ 推出 $s[i] = s[i - p + q]$, 进而 $q - p$ 也是 s 的 period.

Theorem 1.2 (Periodicity Lemma)

若 $p + q - \gcd(p, q) \leq n$, 且 p 和 q 同为 s 的 period, 那么 $\gcd(p, q)$ 也是 s 的 period.

border 与字符串匹配

Theorem 1.3

对于字符串 s 和 t , 若 $2n \geq m$, 则 s 在 t 中的所有匹配位置构成了一个等差数列。

border 与字符串匹配

Theorem 1.3

对于字符串 s 和 t , 若 $2n \geq m$, 则 s 在 t 中的所有匹配位置构成了一个等差数列。

Proof

只需要讨论匹配 3 次及以上的情况。设第 1, 2 次的距离为 u 、第 2, 3 次匹配的距离为 v , 则 $u + v \leq n$, 且 u, v 同为 s 的 period。故 $d = \gcd(u, v)$ 也为 s 的 period。

border 与字符串匹配

Theorem 1.3

对于字符串 s 和 t , 若 $2n \geq m$, 则 s 在 t 中的所有匹配位置构成了一个等差数列。

Proof

只需要讨论匹配 3 次及以上的情况。设第 1, 2 次的距离为 u 、第 2, 3 次匹配的距离为 v , 则 $u + v \leq n$, 且 u, v 同为 s 的 period。故 $d = \gcd(u, v)$ 也为 s 的 period。

如果 $u > d$, 那么第一次匹配往右移 d 位显然又是一个匹配, 故 $u = d$, 余下同理。

border 与字符串匹配

Theorem 1.3

对于字符串 s 和 t , 若 $2n \geq m$, 则 s 在 t 中的所有匹配位置构成了一个等差数列。

Proof

只需要讨论匹配 3 次及以上的情况。设第 1, 2 次的距离为 u 、第 2, 3 次匹配的距离为 v , 则 $u + v \leq n$, 且 u, v 同为 s 的 period。故 $d = \gcd(u, v)$ 也为 s 的 period。

如果 $u > d$, 那么第一次匹配往右移 d 位显然又是一个匹配, 故 $u = d$, 余下同理。

不难发现, 如果匹配次数不小于 3, 则这个等差数列的公差就是 s 的最小 period。

border 的结构

Theorem 1.4

字符串 s 的所有长度 $\geq \frac{n}{2}$ 的 border 长度构成了一个等差数列。

border 的结构

Theorem 1.4

字符串 s 的所有长度 $\geq \frac{n}{2}$ 的 border 长度构成了一个等差数列。

Proof

设 s 的最小 period 为 p , 且 $n - p \geq \frac{n}{2}$ 。考虑另一个 period q , 满足 $n - q \geq \frac{n}{2}$, 则 $p + q \leq n$, 故 q 是 p 的倍数。并且显然所有 p 的倍数都是 s 的 period, 于是得证。

LargePS

Theorem 1.5

设 $|s| = |t| = n$, $\text{LargePS}(s, t)$ 表示满足 $p \geq n/2$ 且 $\text{pre}(s, p) = \text{suf}(t, p)$ 的所有 p , 则 $\text{LargePS}(s, t)$ 组成了一个等差数列。

LargePS

Theorem 1.5

设 $|s| = |t| = n$, $\text{LargePS}(s, t)$ 表示满足 $p \geq n/2$ 且 $\text{pre}(s, p) = \text{suf}(t, p)$ 的所有 p , 则 $\text{LargePS}(s, t)$ 组成了一个等差数列。

Proof

设最大元素为 x 。若 $p < x$ 也在 $\text{LargePS}(s, t)$ 中, 则 $\text{pre}(s, p) = \text{suf}(\text{pre}(s, x), p)$, 所以 p 是 $\text{pre}(s, x)$ 的 border。又 $p \geq n/2 \geq x/2$, 由 Theorem 1.4, 这些 p 构成一个等差数列; 再添上首项 x 仍是等差数列。

border 的分解

Theorem 1.6

字符串 s 的所有 border 长度可以划分成 $\mathcal{O}(\log n)$ 个等差数列。

border 的分解

Theorem 1.6

字符串 s 的所有 border 长度可以划分成 $\mathcal{O}(\log n)$ 个等差数列。

Proof

设 $2^k \leq n < 2^{k+1}$ 。先考虑 $[2^{i-1}, 2^i)$ ，其中 $1 \leq i \leq k$ 。令 $a_i = \text{pre}(s, 2^i)$ ， $b_i = \text{suf}(s, 2^i)$ 。

border 的分解

Theorem 1.6

字符串 s 的所有 border 长度可以划分成 $\mathcal{O}(\log n)$ 个等差数列。

Proof

设 $2^k \leq n < 2^{k+1}$ 。先考虑 $[2^{i-1}, 2^i)$ ，其中 $1 \leq i \leq k$ 。令 $a_i = \text{pre}(s, 2^i)$ ， $b_i = \text{suf}(s, 2^i)$ 。

对任意 $p \in [2^{i-1}, 2^i)$ ，有 $p \geq |a_i|/2$ 。并且 p 是 s 的 border，当且仅当 $\text{pre}(a_i, p) = \text{suf}(b_i, p)$ ，也就是 $p \in \text{LargePS}(a_i, b_i)$ 。

border 的分解

Theorem 1.6

字符串 s 的所有 border 长度可以划分成 $\mathcal{O}(\log n)$ 个等差数列。

Proof

设 $2^k \leq n < 2^{k+1}$ 。先考虑 $[2^{i-1}, 2^i)$ ，其中 $1 \leq i \leq k$ 。令 $a_i = \text{pre}(s, 2^i)$ ， $b_i = \text{suf}(s, 2^i)$ 。

对任意 $p \in [2^{i-1}, 2^i)$ ，有 $p \geq |a_i|/2$ 。并且 p 是 s 的 border，当且仅当 $\text{pre}(a_i, p) = \text{suf}(b_i, p)$ ，也就是 $p \in \text{LargePS}(a_i, b_i)$ 。

因此这一段内的 border 长度，是 $\text{LargePS}(a_i, b_i)$ 与区间 $[2^{i-1}, 2^i)$ 的交。由 Theorem 1.5，它是一个等差数列。

border 的分解

Theorem 1.6

字符串 s 的所有 border 长度可以划分成 $\mathcal{O}(\log n)$ 个等差数列。

Proof

设 $2^k \leq n < 2^{k+1}$ 。先考虑 $[2^{i-1}, 2^i)$ ，其中 $1 \leq i \leq k$ 。令 $a_i = \text{pre}(s, 2^i)$ ， $b_i = \text{suf}(s, 2^i)$ 。

对任意 $p \in [2^{i-1}, 2^i)$ ，有 $p \geq |a_i|/2$ 。并且 p 是 s 的 border，当且仅当 $\text{pre}(a_i, p) = \text{suf}(b_i, p)$ ，也就是 $p \in \text{LargePS}(a_i, b_i)$ 。

因此这一段内的 border 长度，是 $\text{LargePS}(a_i, b_i)$ 与区间 $[2^{i-1}, 2^i)$ 的交。由 Theorem 1.5，它是一个等差数列。

最高段 $[2^k, n)$ 中所有长度都至少为 $n/2$ ，由 Theorem 1.4 也是一个等差数列。这样的长度段只有 $\mathcal{O}(\log n)$ 个。

例题

论战捆竹竿

给定 $s \in \Sigma^n$ 和 w 。初始有一根颜色串为 s 的竹竿。

每次取一根新的 s ，选择 $p \in [0, n]$ ，将新竹竿长度为 p 的前缀与当前竹竿长度为 p 的后缀重合；要求重合位置字符相同。

求所有可达总长度中不超过 w 的不同取值个数。

$T = 5$, $n \leq 5 \times 10^5$, $w \leq 10^{18}$, $\Sigma = \{a, \dots, z\}$ 。

[题目链接](#)

solution

令 P 为所有 period 的集合并上 $\{n\}$, 等价于统计 $[0, w - n]$ 中有多少数能表示为 P 中元素的非负整数线性组合。

solution

令 P 为所有 period 的集合并上 $\{n\}$, 等价于统计 $[0, w - n]$ 中有多少数能表示为 P 中元素的非负整数线性组合。

同余最短路, 考虑能否一次性处理一个等差数列。

solution

令 P 为所有 period 的集合并上 $\{n\}$, 等价于统计 $[0, w - n]$ 中有多少数能表示为 P 中元素的非负整数线性组合。

同余最短路, 考虑能否一次性处理一个等差数列。

考虑处理 $x, x + d, \dots, x + (\ell - 1)d$, 先把状态合并到模 x , 连边后分为 $\gcd(x, d)$ 个环。在每个环上, 转移只来自前面不超过 $\ell - 1$ 个位置, 用单调队列维护 $\text{dis} - d \cdot \text{rank}$ 即可线性松弛。

solution

令 P 为所有 period 的集合并上 $\{n\}$, 等价于统计 $[0, w - n]$ 中有多少数能表示为 P 中元素的非负整数线性组合。

同余最短路, 考虑能否一次性处理一个等差数列。

考虑处理 $x, x + d, \dots, x + (\ell - 1)d$, 先把状态合并到模 x , 连边后分为 $\gcd(x, d)$ 个环。在每个环上, 转移只来自前面不超过 $\ell - 1$ 个位置, 用单调队列维护 $\text{dis} - d \cdot \text{rank}$ 即可线性松弛。

模数变换时, 可以再跑一遍同余最短路维护 dis 。

solution

令 P 为所有 period 的集合并上 $\{n\}$, 等价于统计 $[0, w - n]$ 中有多少数能表示为 P 中元素的非负整数线性组合。

同余最短路, 考虑能否一次性处理一个等差数列。

考虑处理 $x, x + d, \dots, x + (\ell - 1)d$, 先把状态合并到模 x , 连边后分为 $\gcd(x, d)$ 个环。在每个环上, 转移只来自前面不超过 $\ell - 1$ 个位置, 用单调队列维护 $\text{dis} - d \cdot \text{rank}$ 即可线性松弛。

模数变换时, 可以再跑一遍同余最短路维护 dis 。

时间复杂度 $\mathcal{O}(n \log n)$ 。

例题

JOJO

初始 $S = \varepsilon$ 。依次执行 q 次操作：1 $x\ c$ 表示令 $S \leftarrow Sc^x$ ，且保证 $S = \varepsilon$ 或 S 的末字符不是 c ；2 x 表示令 S 回到第 x 次操作后的状态， $x = 0$ 表示空串。

每次操作后输出 $\sum_{i=1}^{|S|} \pi_i \bmod 998244353$ ，其中
 $\pi_i = \max\{p < i \mid \text{pre}(S, p) = \text{suf}(\text{pre}(S, i), p)\}$ 。
 $q \leq 10^5$ ，每个 1 操作中 $x \leq 10^4$ 。

[题目链接](#)

solution

首先考虑没有 2 操作的情况。

solution

首先考虑没有 2 操作的情况。

我们把 (x, c) 看作一个字符，定义两个字符串 $s = s_1 s_2 \cdots s_n$ 和 $t = t_1 t_2 \cdots t_n$ 匹配当且仅当

- ① s_1 的 c 和 t_1 的 c 一样； s_1 的 x 不大于 t_1 的 x 。
- ② 对于 $\forall i \in [2, n]$ ，有 $s_i = t_i$ 。

solution

首先考虑没有 2 操作的情况。

我们把 (x, c) 看作一个字符，定义两个字符串 $s = s_1 s_2 \cdots s_n$ 和 $t = t_1 t_2 \cdots t_n$ 匹配当且仅当

- ① s_1 的 c 和 t_1 的 c 一样； s_1 的 x 不大于 t_1 的 x 。
- ② 对于 $\forall i \in [2, n]$ ，有 $s_i = t_i$ 。

我们按照这样的匹配方式进行 KMP 算法，跳 fail 指针的过程中记录一下最多能匹配多少个字符，比如说已经匹配了 x 个字符，当前可以匹配 y ($y > x$) 个字符，当前的长度为 len ，那么当前结点对答案的贡献就是 $\sum_{i=l+1}^r (len + i)$ 。注意根节点如果出边为当前字符的话需要额外处理剩下的匹配，例如 abaa 中 aa 可以与 a 匹配产生 1 的贡献。

solution

对于可持久化，均摊不能保证复杂度。

solution

对于可持久化，均摊不能保证复杂度。

注意到每次如果 $\text{fail}_i > \frac{i}{2}$ 可以直接令 $i \leftarrow i \bmod d + d$ ，其中 $d = i - \text{fail}_i$ ，因为这一段 border 的下一段字符都是一样的。注意到最长的 border 后面的字符可能不一样，所以需要先跳一次 fail 再这么跳。

solution

对于可持久化，均摊不能保证复杂度。

注意到每次如果 $\text{fail}_i > \frac{i}{2}$ 可以直接令 $i \leftarrow i \bmod d + d$ ，其中 $d = i - \text{fail}_i$ ，因为这一段 border 的下一段字符都是一样的。注意到最长的 border 后面的字符可能不一样，所以需要先跳一次 fail 再这么跳。

即，我们用一个 $\log$ 的代价，使得 KMP 可持久化了。时间复杂度 $\mathcal{O}(n \log n)$ 。

- 1 记号
- 2 border 相关
- 3 ACAM**
- 4 回文串相关
- 5 Z 函数
- 6 后缀数据结构
- 7 基本子串结构
- 8 References

ACAM

给定模式串集合，先把所有模式串插入 Trie。每个结点 u 对应根到 u 的字符串，记为 $\text{str}(u)$ 。

ACAM

给定模式串集合，先把所有模式串插入 Trie。每个结点 u 对应根到 u 的字符串，记为 $\text{str}(u)$ 。

$\text{fail}(u)$ 定义为满足 $\text{str}(v)$ 是 $\text{str}(u)$ 的真后缀的最深结点 v 。这就是 KMP 的失配边在 Trie 上的多模式版本。

ACAM

给定模式串集合，先把所有模式串插入 Trie。每个结点 u 对应根到 u 的字符串，记为 $\text{str}(u)$ 。

$\text{fail}(u)$ 定义为满足 $\text{str}(v)$ 是 $\text{str}(u)$ 的真后缀的最深结点 v 。这就是 KMP 的失配边在 Trie 上的多模式版本。

把不存在的 Trie 转移也补上以后， $\delta(u, c)$ 表示 $\text{str}(u)c$ 的最长 Trie 后缀。于是扫描文本时，当前状态始终是已经读入前缀的最长 Trie 后缀。

ACAM

给定模式串集合，先把所有模式串插入 Trie。每个结点 u 对应根到 u 的字符串，记为 $\text{str}(u)$ 。

$\text{fail}(u)$ 定义为满足 $\text{str}(v)$ 是 $\text{str}(u)$ 的真后缀的最深结点 v 。这就是 KMP 的失配边在 Trie 上的多模式版本。

把不存在的 Trie 转移也补上以后， $\delta(u, c)$ 表示 $\text{str}(u)c$ 的最长 Trie 后缀。于是扫描文本时，当前状态始终是已经读入前缀的最长 Trie 后缀。

fail 和完整转移可以用 BFS 求出。根的儿子们的 fail 为根；处理结点 u 的儿子 $v = \text{ch}(u, c)$ 时，令 $\text{fail}(v) = \delta(\text{fail}(u), c)$ 。若 $\text{ch}(u, c)$ 不存在，就令它等于 $\delta(\text{fail}(u), c)$ ，BFS 顺序保证右边已经被求出。

给定模式串集合，先把所有模式串插入 Trie。每个结点 u 对应根到 u 的字符串，记为 $\text{str}(u)$ 。

$\text{fail}(u)$ 定义为满足 $\text{str}(v)$ 是 $\text{str}(u)$ 的真后缀的最深结点 v 。这就是 KMP 的失配边在 Trie 上的多模式版本。

把不存在的 Trie 转移也补上以后， $\delta(u, c)$ 表示 $\text{str}(u)c$ 的最长 Trie 后缀。于是扫描文本时，当前状态始终是已经读入前缀的最长 Trie 后缀。

fail 和完整转移可以用 BFS 求出。根的儿子 fail 为根；处理结点 u 的儿子 $v = \text{ch}(u, c)$ 时，令 $\text{fail}(v) = \delta(\text{fail}(u), c)$ 。若 $\text{ch}(u, c)$ 不存在，就令它等于 $\delta(\text{fail}(u), c)$ ，BFS 顺序保证右边已经被求出。

扫描文本 t 时, 设读完 $t[1..i]$ 后的状态为 u_i 。所有以 i 为右端点的模式串, 恰好是 u_i 在 fail 链上的终止结点。

例题

A task for substrings

给定互不相同的模式串集合 $S = \{s_1, s_2, \dots, s_N\}$ 、文本串 t 和 Q 次询问 $[l, r]$ 。

对每次询问，求满足 $l \leq a \leq b \leq r$ 且 $t[a..b] \in S$ 的二元组 (a, b) 数量。

$N, Q \leq 5 \times 10^5$, $|t| \leq 5 \times 10^6$, $\sum |s_i| \leq 10^6$ 。

[题目链接](#)

solution

若不要求匹配模式串的左端点在 $[l, r]$ 内，可以直接建出 ACAM、统计 t 的每个前缀有多少后缀是模式串，此时询问变成区间和。

solution

若不要求匹配模式串的左端点在 $[l, r]$ 内，可以直接建出 ACAM、统计 t 的每个前缀有多少后缀是模式串，此时询问变成区间和。

注意到如果一个匹配的右端点在 r 中但是左端点超出范围了，那么这个匹配在 $[l, r]$ 中的部分就是某个模式串的一个后缀。我们考虑能不能预处理出每一个模式串的每一个后缀能匹配的模式串数量。

solution

若不要求匹配模式串的左端点在 $[l, r]$ 内，可以直接建出 ACAM、统计 t 的每个前缀有多少后缀是模式串，此时询问变成区间和。

注意到如果一个匹配的右端点在 r 中但是左端点超出范围了，那么这个匹配在 $[l, r]$ 中的部分就是某个模式串的一个后缀。我们考虑能不能预处理出每一个模式串的每一个后缀能匹配的模式串数量。

可以建反串 ACAM 预处理。

solution

若不要求匹配模式串的左端点在 $[l, r]$ 内, 可以直接建出 ACAM、统计 t 的每个前缀有多少后缀是模式串, 此时询问变成区间和。

注意到如果一个匹配的右端点在 r 中但是左端点超出范围了, 那么这个匹配在 $[l, r]$ 中的部分就是某个模式串的一个后缀。我们考虑能不能预处理出每一个模式串的每一个后缀能匹配的模式串数量。

可以建反串 ACAM 预处理。

实际查询时, 我们需要找到最靠右的 i , 使得存在匹配的右端点为 i , 并且左端点超出范围。这样右端点 $\leq i$ 的匹配我们预处理了; 右端点 $> i$ 的匹配肯定都合法, 用之前的区间和即可,

Scintilla

找这个 i 使用线段树二分即可。时间复杂度 $\mathcal{O}(n \log n)$ 。

例题

Modest Substrings

给定整数区间 $[l, r]$ 和长度 n 。在所有长度为 n 的数字串 a 中，最大化满足 $\text{value}(a[i..j]) \in [l, r]$ 的子串数量；有多解时输出字典序最小的 a 。

含前导零的子串不计入答案，但 a 本身允许有前导零。

$1 \leq l \leq r \leq 10^{800}$, $n \leq 2000$ 。

[题目链接](#)

solution

先把所有值在 $[l, r]$ 内、且不含前导零的十进制串都看成模式串，塞进 ACAM。

solution

先把所有值在 $[l, r]$ 内、且不含前导零的十进制串都看成模式串，塞进 ACAM。
 直接这么做会炸内存，因为 $l, r \leq 10^{800}$ 。但区间 Trie 的结构很规整：沿着 l 和 r 的边界往下走时，只有边界路径上的前缀需要继续分裂；一旦某个前缀已经严格落在 (l, r) 之间，它下面就是一棵完整的十叉树。

solution

先把所有值在 $[l, r]$ 内、且不含前导零的十进制串都看成模式串，塞进 ACAM。直接这么做会炸内存，因为 $l, r \leq 10^{800}$ 。但区间 Trie 的结构很规整：沿着 l 和 r 的边界往下走时，只有边界路径上的前缀需要继续分裂；一旦某个前缀已经严格落在 (l, r) 之间，它下面就是一棵完整的十叉树。

所以不展开这棵子树，而是把它缩成一个点。这个点上挂一个长度标记 k ，表示从这里再走足 k 步，就一定能落到一条合法的叶子路径上。这个标记只有在后面真的还能再走足 k 步时才算；如果串在中途停了，这个贡献就没有了。

solution

先把所有值在 $[l, r]$ 内、且不含前导零的十进制串都看成模式串，塞进 ACAM。直接这么做会炸内存，因为 $l, r \leq 10^{800}$ 。但区间 Trie 的结构很规整：沿着 l 和 r 的边界往下走时，只有边界路径上的前缀需要继续分裂；一旦某个前缀已经严格落在 (l, r) 之间，它下面就是一棵完整的十叉树。

所以不展开这棵子树，而是把它缩成一个点。这个点上挂一个长度标记 k ，表示从这里再走足 k 步，就一定能落到一条合法的叶子路径上。这个标记只有在后面真的还能再走足 k 步时才算；如果串在中途停了，这个贡献就没有了。

一个点可能挂很多不同的 k ，于是我们实际维护的是每个点上每种长度标记的个数。这样压完以后，ACAM 只剩下边界路径和分叉点，规模是 $\mathcal{O}(|l| + |r|)$ 。

solution

对每个点把长度标记做前缀和。这样若当前状态在 u ，并且后面还剩 t 位，就可以直接读出 u 上长度不超过 t 的标记总数。

solution

对每个点把长度标记做前缀和。这样若当前状态在 u ，并且后面还剩 t 位，就可以直接读出 u 上长度不超过 t 的标记总数。

设 $dp[i, u]$ 表示已经填了前 i 位、当前停在状态 u 时的最大答案。转移时枚举下一位 $d \in [0, 9]$ ，走到 $v = \delta(u, d)$ ，再把 v 在剩余位数 $n - i - 1$ 下的贡献加上去，也就是 $dp[i + 1, v] \leftarrow \max\{dp[i + 1, v], dp[i, u] + \text{pref}_v[n - i - 1]\}$ 。

solution

对每个点把长度标记做前缀和。这样若当前状态在 u ，并且后面还剩 t 位，就可以直接读出 u 上长度不超过 t 的标记总数。

设 $dp[i, u]$ 表示已经填了前 i 位、当前停在状态 u 时的最大答案。转移时枚举下一位 $d \in [0, 9]$ ，走到 $v = \delta(u, d)$ ，再把 v 在剩余位数 $n - i - 1$ 下的贡献加上去，也就是 $dp[i + 1, v] \leftarrow \max\{dp[i + 1, v], dp[i, u] + \text{pref}_v[n - i - 1]\}$ 。

这就是标准的 ACAM 上 DP，只是把原来指数级的模式串集合压成了少量带长度标记的点。建树时按数位 DP 的方式递归：长度等于边界时沿着两条边界路径走，长度严格夹在中间时整层直接压成完整子树。

solution

对每个点把长度标记做前缀和。这样若当前状态在 u ，并且后面还剩 t 位，就可以直接读出 u 上长度不超过 t 的标记总数。

设 $dp[i, u]$ 表示已经填了前 i 位、当前停在状态 u 时的最大答案。转移时枚举下一位 $d \in [0, 9]$ ，走到 $v = \delta(u, d)$ ，再把 v 在剩余位数 $n - i - 1$ 下的贡献加上去，也就是 $dp[i + 1, v] \leftarrow \max\{dp[i + 1, v], dp[i, u] + \text{pref}_v[n - i - 1]\}$ 。

这就是标准的 ACAM 上 DP，只是把原来指数级的模式串集合压成了少量带长度标记的点。建树时按数位 DP 的方式递归：长度等于边界时沿着两条边界路径走，长度严格夹在中间时整层直接压成完整子树。

输出方案时，再倒着标记哪些状态还能走到最优值，记成 $F[i, u]$ ；正着输出时，每一位都贪心取最小的可行数字即可。总复杂度是 $\mathcal{O}(10n(|l| + |r|))$ 。

例题

novel

给定 N 个字符串 $S_1, S_2, \dots, S_N$ 及权值 $w_1, w_2, \dots, w_N$ 。设 $f(S, T)$ 为 T 在 S 中的出现次数， $g(S) = \sum_{i=1}^N w_i f(S, S_i)$ 。

对非空串 S ，定义 $h(S) = \max_{1 \leq l \leq r \leq |S|} \frac{g(S[l..r])}{r-l+1}$ 。求所有 $h(S_i[1..j])$ 。

$N \leq 2 \times 10^5$ ， $\sum |S_i| \leq 5 \times 10^6$ ， $\Sigma = \{a, b, c, d\}$ 。

[题目链接](#)

solution

先看一个源串 S 的所有前缀。对所有模式串建 ACAM, 令 $W(u)$ 为 u 的 fail 链上所有终止模式串的权值和; 扫描到状态 u 时, $W(u)$ 就是以当前位置结尾的新增权值。

solution

先看一个源串 S 的所有前缀。对所有模式串建 ACAM, 令 $W(u)$ 为 u 的 fail 链上所有终止模式串的权值和; 扫描到状态 u 时, $W(u)$ 就是以当前位置结尾的新增权值。

设当前右端点为 r , 并记 $z_l = g(S[l..r])$ 。要求 $\max_l \frac{z_l}{r-l+1}$, 等价于在点 $(l, -z_l)$ 的上凸壳上查询与点 $(r+1, 0)$ 的最大斜率。

solution

先看一个源串 S 的所有前缀。对所有模式串建 ACAM, 令 $W(u)$ 为 u 的 fail 链上所有终止模式串的权值和; 扫描到状态 u 时, $W(u)$ 就是以当前位置结尾的新增权值。

设当前右端点为 r , 并记 $z_l = g(S[l..r])$ 。要求 $\max_l \frac{z_l}{r-l+1}$, 等价于在点 $(l, -z_l)$ 的上凸壳上查询与点 $(r+1, 0)$ 的最大斜率。

当 r 右移一位时, 大多数 z_l 都会加上同一个以新位置结尾的匹配权值。这个全局加法只改变所有点的纵坐标偏移, 可以懒标记维护。

solution

先看一个源串 S 的所有前缀。对所有模式串建 ACAM，令 $W(u)$ 为 u 的 fail 链上所有终止模式串的权值和；扫描到状态 u 时， $W(u)$ 就是以当前位置结尾的新增权值。

设当前右端点为 r ，并记 $z_l = g(S[l..r])$ 。要求 $\max_l \frac{z_l}{r-l+1}$ ，等价于在点 $(l, -z_l)$ 的上凸壳上查询与点 $(r+1, 0)$ 的最大斜率。

当 r 右移一位时，大多数 z_l 都会加上同一个以新位置结尾的匹配权值。这个全局加法只改变所有点的纵坐标偏移，可以懒标记维护。

真正麻烦的是最近的若干左端点，因为它们对应的短串还没有稳定到同一个 ACAM 状态；朴素做法要把短后缀反过来在反串 ACAM 上走一遍。

solution

先看一个源串 S 的所有前缀。对所有模式串建 ACAM，令 $W(u)$ 为 u 的 fail 链上所有终止模式串的权值和；扫描到状态 u 时， $W(u)$ 就是以当前位置结尾的新增权值。

设当前右端点为 r ，并记 $z_l = g(S[l..r])$ 。要求 $\max_l \frac{z_l}{r-l+1}$ ，等价于在点 $(l, -z_l)$ 的上凸壳上查询与点 $(r+1, 0)$ 的最大斜率。

当 r 右移一位时，大多数 z_l 都会加上同一个以新位置结尾的匹配权值。这个全局加法只改变所有点的纵坐标偏移，可以懒标记维护。

真正麻烦的是最近的若干左端点，因为它们对应的短串还没有稳定到同一个 ACAM 状态；朴素做法要把短后缀反过来在反串 ACAM 上走一遍。

设最长模式串长度为 L ，总输入长度为 m 。若只按上面的想法维护，一个前缀每次扩展时需要特殊处理 $O(L)$ 个短后缀，于是得到一个 $O(mL)$ 的算法。

solution

先看一个源串 S 的所有前缀。对所有模式串建 ACAM，令 $W(u)$ 为 u 的 fail 链上所有终止模式串的权值和；扫描到状态 u 时， $W(u)$ 就是以当前位置结尾的新增权值。

设当前右端点为 r ，并记 $z_l = g(S[l..r])$ 。要求 $\max_l \frac{z_l}{r-l+1}$ ，等价于在点 $(l, -z_l)$ 的上凸壳上查询与点 $(r+1, 0)$ 的最大斜率。

当 r 右移一位时，大多数 z_l 都会加上同一个以新位置结尾的匹配权值。这个全局加法只改变所有点的纵坐标偏移，可以懒标记维护。

真正麻烦的是最近的若干左端点，因为它们对应的短串还没有稳定到同一个 ACAM 状态；朴素做法要把短后缀反过来在反串 ACAM 上走一遍。

设最长模式串长度为 L ，总输入长度为 m 。若只按上面的想法维护，一个前缀每次扩展时需要特殊处理 $O(L)$ 个短后缀，于是得到一个 $O(mL)$ 的算法。

瓶颈就在于“把短后缀反过来喂进 ACAM”这一段。关键性质是：对于 ACAM 上的结点 u ，长度在 $(|\text{fail}(u)|, |u|]$ 内的后缀，在自动机上匹配后最终都会落在 u 负责的同一状态里；更短的后缀已经归入 $\text{fail}(u)$ 。

solution

因此对每个结点 u ，维护长度在 $(|\text{fail}(u)|, |u|]$ 内的后缀所能得到的最优凸壳信息。加一个字符以后，除了代表 u 自身的最长后缀，其余后缀的转移和增量都是相同的。

solution

因此对每个结点 u ，维护长度在 $(|\text{fail}(u)|, |u|]$ 内的后缀所能得到的最优凸壳信息。加一个字符以后，除了代表 u 自身的最长后缀，其余后缀的转移和增量都是相同的。

这样同一段后缀可以整段加、整段插入新点，并用双指针在凸壳上查询斜率最大值。

solution

因此对每个结点 u , 维护长度在 $(|\text{fail}(u)|, |u|]$ 内的后缀所能得到的最优凸壳信息。加一个字符以后, 除了代表 u 自身的最长后缀, 其余后缀的转移和增量都是相同的。

这样同一段后缀可以整段加、整段插入新点, 并用双指针在凸壳上查询斜率最大值。

按 Trie 的 BFS 序处理所有前缀。设当前前缀对应结点为 u , 它的 Trie 父亲表示删掉最后一个字符以前的前缀, fail 父亲表示更短的一组后缀。当前结点的答案由父亲前缀继承来的凸壳、 $\text{fail}(u)$ 负责的更短后缀, 以及 u 自己这段长度区间中新加入的最长后缀共同更新。

solution

因此对每个结点 u ，维护长度在 $(|\text{fail}(u)|, |u|]$ 内的后缀所能得到的最优凸壳信息。加一个字符以后，除了代表 u 自身的最长后缀，其余后缀的转移和增量都是相同的。

这样同一段后缀可以整段加、整段插入新点，并用双指针在凸壳上查询斜率最大值。

按 Trie 的 BFS 序处理所有前缀。设当前前缀对应结点为 u ，它的 Trie 父亲表示删掉最后一个字符以前的前缀，fail 父亲表示更短的一组后缀。当前结点的答案由父亲前缀继承来的凸壳、fail(u) 负责的更短后缀，以及 u 自己这段长度区间中新加入的最长后缀共同更新。

还需要快速知道某个前缀的短后缀反过来后在反串 ACAM 中的位置。注意这些串一定是某个模式串前缀的反串，所以可以在建两棵 ACAM 时一并预处理。

这样同一段后缀可以整段加、整段插入新点，并用双指针在凸壳上查询斜率最大值。

还需要快速知道某个前缀的短后缀反过来后在反串 ACAM 中的位置。注意这些串一定是某个模式串前缀的反串，所以可以在建两棵 ACAM 时一并预处理。

字符集大小为 4，每条 Trie 边、fail 边以及每次凸壳插入/查询都只被处理常数次数。总复杂度为 $\mathcal{O}(\sum |S_i|)$ 。

- 1 记号
- 2 border 相关
- 3 ACAM
- 4 回文串相关**
- 5 Z 函数
- 6 后缀数据结构
- 7 基本子串结构
- 8 References

Manacher

对原串两端及相邻字符之间插入同一个分隔符，这样原串的奇回文和偶回文都会变成 t 中的奇回文。

Manacher

对原串两端及相邻字符之间插入同一个分隔符，这样原串的奇回文和偶回文都会变成 t 中的奇回文。

对 t 的每个位置 i ，记 d_i 为以 i 为中心的最大回文半径，也就是最大的 d_i 使得 $t[i - d_i + 1..i + d_i - 1]$ 是回文串。

Manacher

对原串两端及相邻字符之间插入同一个分隔符，这样原串的奇回文和偶回文都会变成 t 中的奇回文。

对 t 的每个位置 i ，记 d_i 为以 i 为中心的最大回文半径，也就是最大的 d_i 使得 $t[i - d_i + 1..i + d_i - 1]$ 是回文串。

只要知道 d_i ，一个原串区间是不是回文、某个中心的最大偶回文半径、最长回文子串等信息，都可以由中心和半径直接换算回来。

Manacher

从左到右求 d_i 。算法始终维护当前已经得到的、右端点最大的回文区间 $[L, R]$ 。

Manacher

从左到右求 d_i 。算法始终维护当前已经得到的、右端点最大的回文区间 $[L, R]$ 。

若 $i > R$ ，则当前中心还没有被任何已知回文覆盖，只能令 $d_i \leftarrow 1$ 后向两侧暴力扩展。

Manacher

从左到右求 d_i 。算法始终维护当前已经得到的、右端点最大的回文区间 $[L, R]$ 。

若 $i > R$ ，则当前中心还没有被任何已知回文覆盖，只能令 $d_i \leftarrow 1$ 后向两侧暴力扩展。

若 $i \leq R$ ，令 $j = L + R - i$ 为 i 关于 $[L, R]$ 中心的对称点。区间内部的对称性保证 $d_i \geq \min(d_j, R - i + 1)$ ，于是先用这个值作为初始半径。

Manacher

从左到右求 d_i 。算法始终维护当前已经得到的、右端点最大的回文区间 $[L, R]$ 。

若 $i > R$ ，则当前中心还没有被任何已知回文覆盖，只能令 $d_i \leftarrow 1$ 后向两侧暴力扩展。

若 $i \leq R$ ，令 $j = L + R - i$ 为 i 关于 $[L, R]$ 中心的对称点。区间内部的对称性保证 $d_i \geq \min(d_j, R - i + 1)$ ，于是先用这个值作为初始半径。

只有当初始回文恰好顶到右边界 R 时，才可能继续向外扩展；否则 d_i 已经完全由 d_j 决定。

Manacher

每次求出 d_i 后, 若 $i + d_i - 1 > R$, 就用新的回文区间更新 L, R 。

Manacher

每次求出 d_i 后，若 $i + d_i - 1 > R$ ，就用新的回文区间更新 L, R 。
复杂度证明只看暴力比较：一次成功扩展会使右端点 R 至少右移一位；不成功的比较每个中心至多发生一次。

Manacher

每次求出 d_i 后, 若 $i + d_i - 1 > R$, 就用新的回文区间更新 L, R 。

复杂度证明只看暴力比较: 一次成功扩展会使右端点 R 至少右移一位; 不成功的比较每个中心至多发生一次。

因此所有中心上的总扩展次数是 $\mathcal{O}(|t|)$, Manacher 在线性时间内求出所有半径。

PAM

PAM (Palindromic Tree) 的每个非根结点对应一个本质不同回文子串。结点 u 记录长度 $\text{len}(u)$ ，以及在两端加入同一个字符后的转移。

PAM

PAM (Palindromic Tree) 的每个非根结点对应一个本质不同回文子串。结点 u 记录长度 $\text{len}(u)$ ，以及在两端加入同一个字符后的转移。

$\text{fail}(u)$ 指向 u 的最长真回文后缀。为了同时处理奇偶长度，建立两个根：奇根长度为 -1 ，偶根长度为 0 ，并令偶根的 fail 指向奇根。

PAM

PAM (Palindromic Tree) 的每个非根结点对应一个本质不同回文子串。结点 u 记录长度 $\text{len}(u)$ ，以及在两端加入同一个字符后的转移。

$\text{fail}(u)$ 指向 u 的最长真回文后缀。为了同时处理奇偶长度，建立两个根：奇根长度为 -1 ，偶根长度为 0 ，并令偶根的 fail 指向奇根。

处理到位置 p 后，指针 last 表示当前前缀的最长回文后缀。沿 last 的 fail 链向上走，恰好枚举当前前缀的所有回文后缀。

PAM

向右加入字符 s_p 时, 从 $last$ 开始沿 fail 跳, 直到找到第一个结点 u 满足 $s_{p-\text{len}(u)-1} = s_p$ 。

PAM

向右加入字符 s_p 时, 从 $last$ 开始沿 fail 跳, 直到找到第一个结点 u 满足 $s_{p-\text{len}(u)-1} = s_p$ 。

若 u 已经有字符 s_p 的转移, 那么新前缀的最长回文后缀就是这个转移指向的结点。

PAM

向右加入字符 s_p 时，从 $last$ 开始沿 fail 跳，直到找到第一个结点 u 满足 $s_{p-\text{len}(u)-1} = s_p$ 。

若 u 已经有字符 s_p 的转移，那么新前缀的最长回文后缀就是这个转移指向的结点。

否则新建结点 v ，令 $\text{len}(v) = \text{len}(u) + 2$ ，并把 u 的 s_p 转移连到 v 。

PAM

向右加入字符 s_p 时，从 $last$ 开始沿 fail 跳，直到找到第一个结点 u 满足 $s_{p-\text{len}(u)-1} = s_p$ 。

若 u 已经有字符 s_p 的转移，那么新前缀的最长回文后缀就是这个转移指向的结点。

否则新建结点 v ，令 $\text{len}(v) = \text{len}(u) + 2$ ，并把 u 的 s_p 转移连到 v 。

v 的 fail 也用同样的方法求：从 $\text{fail}(u)$ 开始继续跳，找到下一个能被 s_p 包住回文后缀，其对应转移就是 $\text{fail}(v)$ 。

PAM

每加入一个字符至多新建一个结点，因为新增的本质不同回文串只能是当前前缀的最长回文后缀。

PAM

每加入一个字符至多新建一个结点，因为新增的本质不同回文串只能是当前前缀的最长回文后缀。

fail 跳转总复杂度均摊线性。

PAM

每加入一个字符至多新建一个结点，因为新增的本质不同回文串只能是当前前缀的最长回文后缀。

fail 跳转总复杂度均摊线性。

若要统计每个回文的出现次数，可以先在每个位置给 *last* 加一，最后按 *len* 从大到小把计数累加到 fail。

PAM 的 series link

对结点 u 定义 $\text{diff}(u) = \text{len}(u) - \text{len}(\text{fail}(u))$ 。

PAM 的 series link

对结点 u 定义 $\text{diff}(u) = \text{len}(u) - \text{len}(\text{fail}(u))$ 。

若 $u, \text{fail}(u), \text{fail}^2(u), \dots$ 上连续若干个结点的 diff 相同，那么它们的长度构成同公差等差数列。

PAM 的 series link

对结点 u 定义 $\text{diff}(u) = \text{len}(u) - \text{len}(\text{fail}(u))$ 。

若 $u, \text{fail}(u), \text{fail}^2(u), \dots$ 上连续若干个结点的 diff 相同，那么它们的长度构成同公差等差数列。

令 $\text{slink}(u)$ 跳到 fail 链上第一个 diff 改变的位置。于是枚举所有回文后缀时，只需要逐段跳 series link 。

PAM 的 series link

对结点 u 定义 $\text{diff}(u) = \text{len}(u) - \text{len}(\text{fail}(u))$ 。

若 $u, \text{fail}(u), \text{fail}^2(u), \dots$ 上连续若干个结点的 diff 相同，那么它们的长度构成同公差等差数列。

令 $\text{slink}(u)$ 跳到 fail 链上第一个 diff 改变的位置。于是枚举所有回文后缀时，只需要逐段跳 series link。

一条 fail 链会被划分成 $\mathcal{O}(\log n)$ 个这样的等差段。

例题

字符串

给定 $s \in \Sigma^n$ 和 q 次询问 (i, r) 。

对每次询问，求满足 $1 \leq l \leq r$ 且
 $s[i..i+l-1] < (s[i+l..i+2l-1])^R$ 的 l 的数量。

$T \leq 5$, $n, q \leq 10^5$, $i+2r-1 \leq n$, $\Sigma = \{a, \dots, z\}$ 。

[题目链接](#)

solution

先处理没有偶回文的情形。令 $T = ss^R$ 并建立后缀数组。

solution

先处理没有偶回文的情形。令 $T = ss^R$ 并建立后缀数组。

比较 $s[i..i+l-1]$ 和 $(s[i+l..i+2l-1])^R$, 可以先放宽成比较两个后缀:
 $T[i..]$ 与 $T[2n-i-2l+2..]$ 。

solution

先处理没有偶回文的情形。令 $T = ss^R$ 并建立后缀数组。

比较 $s[i..i+l-1]$ 和 $(s[i+l..i+2l-1])^R$, 可以先放宽成比较两个后缀:
 $T[i..]$ 与 $T[2n-i-2l+2..]$ 。

若 $s[i..i+2l-1]$ 不是偶回文, 则第一次不同一定发生在这 l 位以内, 放宽比较与原比较等价。

solution

先处理没有偶回文的情形。令 $T = ss^R$ 并建立后缀数组。

比较 $s[i..i+l-1]$ 和 $(s[i+l..i+2l-1])^R$, 可以先放宽成比较两个后缀:
 $T[i..]$ 与 $T[2n-i-2l+2..]$ 。

若 $s[i..i+2l-1]$ 不是偶回文, 则第一次不同一定发生在这 l 位以内, 放宽比较与原比较等价。

对一个询问 (i, r) , 令 $p = 2n - i - 2l + 2$ 。需要统计 $p \in [2n - i - 2r + 2, 2n - i]$,
 $p \equiv i \pmod{2}$, 且 $\text{rk}_p > \text{rk}_i$ 。这是一个按位置和后缀排名的二维数点。

solution

先处理没有偶回文的情形。令 $T = ss^R$ 并建立后缀数组。

比较 $s[i..i+l-1]$ 和 $(s[i+l..i+2l-1])^R$, 可以先放宽成比较两个后缀:
 $T[i..]$ 与 $T[2n-i-2l+2..]$ 。

若 $s[i..i+2l-1]$ 不是偶回文, 则第一次不同一定发生在这 l 位以内, 放宽比较与原比较等价。

对一个询问 (i, r) , 令 $p = 2n - i - 2l + 2$ 。需要统计 $p \in [2n - i - 2r + 2, 2n - i]$,
 $p \equiv i \pmod{2}$, 且 $\text{rk}_p > \text{rk}_i$ 。这是一个按位置和后缀排名的二维数点。

因此可以先用主席树或离线树状数组求出放宽后的答案。剩下只需减掉偶回文导致的错误贡献。

solution

先处理没有偶回文的情形。令 $T = ss^R$ 并建立后缀数组。

比较 $s[i..i+l-1]$ 和 $(s[i+l..i+2l-1])^R$, 可以先放宽成比较两个后缀:
 $T[i..]$ 与 $T[2n-i-2l+2..]$ 。

若 $s[i..i+2l-1]$ 不是偶回文, 则第一次不同一定发生在这 l 位以内, 放宽比较与原比较等价。

对一个询问 (i, r) , 令 $p = 2n - i - 2l + 2$ 。需要统计 $p \in [2n - i - 2r + 2, 2n - i]$,
 $p \equiv i \pmod{2}$, 且 $\text{rk}_p > \text{rk}_i$ 。这是一个按位置和后缀排名的二维数点。

因此可以先用主席树或离线树状数组求出放宽后的答案。剩下只需减掉偶回文导致的错误贡献。

用 Manacher 求每个偶回文中心 c 的最大半径, 记它的极长回文区间为 $[L_c, R_c]$ 。所有被这个极长回文包含且中心同为 c 的偶回文, 前 l 位比较都会完全相等。

solution

放宽比较会继续越过中点，真正决定大小的是极长回文两侧第一次不同的位置。于是对每个中心 c ，只需用后缀数组比较一次回文右侧 $s[R_c + 1..]$ 与左侧反串对应后缀。

solution

放宽比较会继续越过中点，真正决定大小的是极长回文两侧第一次不同的位置。于是对每个中心 c ，只需用后缀数组比较一次回文右侧 $s[R_c + 1..]$ 与左侧反串对应后缀。

若这次比较说明放宽答案把该中心的偶回文算成了合法，则这个中心下所有被包含的偶回文都会产生同一种多算，需要统一扣除。

solution

放宽比较会继续越过中点，真正决定大小的是极长回文两侧第一次不同的位置。于是对每个中心 c ，只需用后缀数组比较一次回文右侧 $s[R_c + 1..]$ 与左侧反串对应后缀。

若这次比较说明放宽答案把该中心的偶回文算成了合法，则这个中心下所有被包含的偶回文都会产生同一种多算，需要统一扣除。

把中心 c 看成位于 c 与 $c+1$ 之间，最大半径为 ρ_c ，则它能产生的左端点 i 满足 $c - \rho_c + 1 \leq i \leq c$ 。对询问 (i, r) ，选择中心 c 对应的长度为 $l = c - i + 1$ ，还要求 $1 \leq l \leq r$ ，也就是 $i \leq c \leq i + r - 1$ 。

solution

放宽比较会继续越过中点，真正决定大小的是极长回文两侧第一次不同的位置。于是对每个中心 c ，只需用后缀数组比较一次回文右侧 $s[R_c + 1..]$ 与左侧反串对应后缀。

若这次比较说明放宽答案把该中心的偶回文算成了合法，则这个中心下所有被包含的偶回文都会产生同一种多算，需要统一扣除。

把中心 c 看成位于 c 与 $c+1$ 之间，最大半径为 ρ_c ，则它能产生的左端点 i 满足 $c - \rho_c + 1 \leq i \leq c$ 。对询问 (i, r) ，选择中心 c 对应的长度为 $l = c - i + 1$ ，还要求 $1 \leq l \leq r$ ，也就是 $i \leq c \leq i + r - 1$ 。

因而一个会多算的中心 c 对所有左端点 $i \in [c - \rho_c + 1, c]$ 做一次区间加。将询问按右界 $i + r - 1$ 排序，按 c 从左到右加入需要扣除的中心，用树状数组做区间加、单点查即可。总复杂度 $\mathcal{O}((n + q) \log n)$ 。

例题

Reverses

给定 $s, t \in \Sigma^n$ 。选择若干个两两不交的区间 $[l_j, r_j]$ ，把 $t[l_j..r_j]$ 各自翻转，要求最终得到 s 。

求区间数量的最小值，并输出任意一组最优区间；若无解，输出 -1 。

$n \leq 5 \times 10^5$, $\Sigma = \{a, \dots, z\}$ 。

[题目链接](#)

solution

一个区间 $[l, r]$ 被翻转后正确，当且仅当 $s[l..r] = (t[l..r])^R$ 。这类条件可以把两串交错起来处理。

solution

一个区间 $[l, r]$ 被翻转后正确，当且仅当 $s[l..r] = (t[l..r])^R$ 。这类条件可以把两串交错起来处理。

令 $u = s_1 t_1 s_2 t_2 \cdots s_n t_n$ 。那么 $[l, r]$ 可作为一个翻转区间，当且仅当 $u[2l-1..2r]$ 是回文。

solution

一个区间 $[l, r]$ 被翻转后正确, 当且仅当 $s[l..r] = (t[l..r])^R$ 。这类条件可以把两串交错起来处理。

令 $u = s_1 t_1 s_2 t_2 \cdots s_n t_n$ 。那么 $[l, r]$ 可作为一个翻转区间, 当且仅当 $u[2l-1..2r]$ 是回文。

因而问题变成最小偶回文划分。设 f_i 为处理完前 i 个原位置的最少操作数。若 $s_i = t_i$, 位置 i 不需要翻转, 可以用 f_{i-1} 更新 f_i 。

solution

一个区间 $[l, r]$ 被翻转后正确, 当且仅当 $s[l..r] = (t[l..r])^R$ 。这类条件可以把两串交错起来处理。

令 $u = s_1 t_1 s_2 t_2 \cdots s_n t_n$ 。那么 $[l, r]$ 可作为一个翻转区间, 当且仅当 $u[2l-1..2r]$ 是回文。

因而问题变成最小偶回文划分。设 f_i 为处理完前 i 个原位置的最少操作数。若 $s_i = t_i$, 位置 i 不需要翻转, 可以用 f_{i-1} 更新 f_i 。

若 $u[2j-1..2i]$ 是回文且 $j < i$, 则区间 $[j, i]$ 可以作为一次翻转, 得到 $f_i \leftarrow f_{j-1} + 1$ 。长度为 2 的回文对应 $j = i$, 它不是一次操作, 需要跳过。

solution

一个区间 $[l, r]$ 被翻转后正确, 当且仅当 $s[l..r] = (t[l..r])^R$ 。这类条件可以把两串交错起来处理。

令 $u = s_1 t_1 s_2 t_2 \cdots s_n t_n$ 。那么 $[l, r]$ 可作为一个翻转区间, 当且仅当 $u[2l-1..2r]$ 是回文。

因而问题变成最小偶回文划分。设 f_i 为处理完前 i 个原位置的最少操作数。若 $s_i = t_i$, 位置 i 不需要翻转, 可以用 f_{i-1} 更新 f_i 。

若 $u[2j-1..2i]$ 是回文且 $j < i$, 则区间 $[j, i]$ 可以作为一次翻转, 得到 $f_i \leftarrow f_{j-1} + 1$ 。长度为 2 的回文对应 $j = i$, 它不是一次操作, 需要跳过。

需要快速枚举每个前缀 $u[1..2i]$ 的回文后缀。建立 u 的 PAM, 处理到位置 $2i$ 时, $last$ 的 fail 链正好给出所有候选。

solution

一个区间 $[l, r]$ 被翻转后正确, 当且仅当 $s[l..r] = (t[l..r])^R$ 。这类条件可以把两串交错起来处理。

令 $u = s_1 t_1 s_2 t_2 \cdots s_n t_n$ 。那么 $[l, r]$ 可作为一个翻转区间, 当且仅当 $u[2l-1..2r]$ 是回文。

因而问题变成最小偶回文划分。设 f_i 为处理完前 i 个原位置的最少操作数。若 $s_i = t_i$, 位置 i 不需要翻转, 可以用 f_{i-1} 更新 f_i 。

若 $u[2j-1..2i]$ 是回文且 $j < i$, 则区间 $[j, i]$ 可以作为一次翻转, 得到 $f_i \leftarrow f_{j-1} + 1$ 。长度为 2 的回文对应 $j = i$, 它不是一次操作, 需要跳过。

需要快速枚举每个前缀 $u[1..2i]$ 的回文后缀。建立 u 的 PAM, 处理到位置 $2i$ 时, $last$ 的 fail 链正好给出所有候选。

直接跳 fail 会退化到平方。定义 $\text{diff}(v) = \text{len}(v) - \text{len}(\text{fail}(v))$ 。fail 链上连续 diff 相同的一段, 其回文长度构成等差数列。

solution

用 series link 跳到下一段。对于同一段中的所有回文，左端 j 也按同一公差变化，所有转移都是 $f_i \leftarrow f_{j-1} + 1$ 。

solution

用 series link 跳到下一段。对于同一段中的所有回文，左端 j 也按同一公差变化，所有转移都是 $f_i \leftarrow f_{j-1} + 1$ 。

因此每个 series 段只需要维护这一段里最小的 f_{j-1} 以及对应的 j 。

solution

用 series link 跳到下一段。对于同一段中的所有回文，左端 j 也按同一公差变化，所有转移都是 $f_i \leftarrow f_{j-1} + 1$ 。

因此每个 series 段只需要维护这一段里最小的 f_{j-1} 以及对应的 j 。

对一个回文结点 v ，当前右端点为 $2i$ 时，它对应的翻转区间左端为 $j = i - \text{len}(v)/2 + 1$ 。长度为 2 的回文对应 $j = i$ ，不能作为一次翻转，需要跳过。

solution

用 series link 跳到下一段。对于同一段中的所有回文，左端 j 也按同一公差变化，所有转移都是 $f_i \leftarrow f_{j-1} + 1$ 。

因此每个 series 段只需要维护这一段里最小的 f_{j-1} 以及对应的 j 。

对一个回文结点 v ，当前右端点为 $2i$ 时，它对应的翻转区间左端为 $j = i - \text{len}(v)/2 + 1$ 。长度为 2 的回文对应 $j = i$ ，不能作为一次翻转，需要跳过。

处理 v 时，先用段首对应的 j 初始化临时最小值。若 $\text{fail}(v)$ 与 v 仍在同一段，就用父亲维护好的最小值更新它；正确性来自同段祖先在之前相应位置已经被访问并维护过。

solution

用 series link 跳到下一段。对于同一段中的所有回文，左端 j 也按同一公差变化，所有转移都是 $f_i \leftarrow f_{j-1} + 1$ 。

因此每个 series 段只需要维护这一段里最小的 f_{j-1} 以及对应的 j 。

对一个回文结点 v ，当前右端点为 $2i$ 时，它对应的翻转区间左端为 $j = i - \text{len}(v)/2 + 1$ 。长度为 2 的回文对应 $j = i$ ，不能作为一次翻转，需要跳过。

处理 v 时，先用段首对应的 j 初始化临时最小值。若 $\text{fail}(v)$ 与 v 仍在同一段，就用父亲维护好的最小值更新它；正确性来自同段祖先在之前相应位置已经被访问并维护过。

每个位置只访问 $\mathcal{O}(\log n)$ 个 series 段。若最后 $f_n = +\infty$ 则无解；否则记录来源并从 n 倒推即可恢复区间。复杂度 $\mathcal{O}(n \log n)$ 。

例题

I Love CCPC

称串 x 为 CCPC 串，当且仅当 $|x| \geq 4$, $x_1 = x_2 = x_{|x|}$, 且 $x[3..|x| - 1]$ 是回文串。

给定初始串 s , 随后每次在当前串左端或右端插入一个字符。每次插入后, 求当前串中 CCPC 子串的数量, 按区间计数。

$T \leq 100$, $\sum(|s| + q) \leq 5 \times 10^5$, $\Sigma = \{a, \dots, z\}$ 。

[题目链接](#)

solution

先把所有操作离线，得到最终大串 S 。任意时刻当前串都是 $S[l..r]$ ；一次左插入就是 $l \leftarrow l - 1$ ，一次右插入就是 $r \leftarrow r + 1$ 。

solution

先把所有操作离线，得到最终大串 S 。任意时刻当前串都是 $S[l..r]$ ；一次左插入就是 $l \leftarrow l - 1$ ，一次右插入就是 $r \leftarrow r + 1$ 。

这样每次操作只会新增以新端点为端点的 CCPC 子串，旧区间内部的贡献完全不变。下面只讨论如何计算一次扩展带来的增量。

solution

先把所有操作离线，得到最终大串 S 。任意时刻当前串都是 $S[l..r]$ ；一次左插入就是 $l \leftarrow l - 1$ ，一次右插入就是 $r \leftarrow r + 1$ 。

这样每次操作只会新增以新端点为端点的 CCPC 子串，旧区间内部的贡献完全不变。下面只讨论如何计算一次扩展带来的增量。

若向右扩展到 $R = r + 1$ ，新增子串一定形如 $S[p..R]$ 。由定义，它合法当且仅当 $p \leq R - 3$ ， $S[p] = S[p + 1] = S[R]$ ，且 $S[p + 1..R]$ 是回文串。

solution

先把所有操作离线，得到最终大串 S 。任意时刻当前串都是 $S[l..r]$ ；一次左插入就是 $l \leftarrow l - 1$ ，一次右插入就是 $r \leftarrow r + 1$ 。

这样每次操作只会新增以新端点为端点的 CCPC 子串，旧区间内部的贡献完全不变。下面只讨论如何计算一次扩展带来的增量。

若向右扩展到 $R = r + 1$ ，新增子串一定形如 $S[p..R]$ 。由定义，它合法当且仅当 $p \leq R - 3$ ， $S[p] = S[p + 1] = S[R]$ ，且 $S[p + 1..R]$ 是回文串。

因此需要统计前缀 $S[1..R]$ 的回文后缀 $S[q..R]$ ，其中 $q = p + 1$ ，满足 $l + 1 \leq q \leq R - 2$ 且 $S[q - 1] = S[q]$ 。

solution

先把所有操作离线，得到最终大串 S 。任意时刻当前串都是 $S[l..r]$ ；一次左插入就是 $l \leftarrow l - 1$ ，一次右插入就是 $r \leftarrow r + 1$ 。

这样每次操作只会新增以新端点为端点的 CCPC 子串，旧区间内部的贡献完全不变。下面只讨论如何计算一次扩展带来的增量。

若向右扩展到 $R = r + 1$ ，新增子串一定形如 $S[p..R]$ 。由定义，它合法当且仅当 $p \leq R - 3$ ， $S[p] = S[p + 1] = S[R]$ ，且 $S[p + 1..R]$ 是回文串。

因此需要统计前缀 $S[1..R]$ 的回文后缀 $S[q..R]$ ，其中 $q = p + 1$ ，满足 $l + 1 \leq q \leq R - 2$ 且 $S[q - 1] = S[q]$ 。

对 S 建 PAM。设 v_R 为处理到位置 R 时的 *last*，那么 v_R 的 fail 祖先恰好对应所有以 R 结尾的回文后缀。

solution

先把所有操作离线，得到最终大串 S 。任意时刻当前串都是 $S[l..r]$ ；一次左插入就是 $l \leftarrow l - 1$ ，一次右插入就是 $r \leftarrow r + 1$ 。

这样每次操作只会新增以新端点为端点的 CCPC 子串，旧区间内部的贡献完全不变。下面只讨论如何计算一次扩展带来的增量。

若向右扩展到 $R = r + 1$ ，新增子串一定形如 $S[p..R]$ 。由定义，它合法当且仅当 $p \leq R - 3$ ， $S[p] = S[p + 1] = S[R]$ ，且 $S[p + 1..R]$ 是回文串。

因此需要统计前缀 $S[1..R]$ 的回文后缀 $S[q..R]$ ，其中 $q = p + 1$ ，满足 $l + 1 \leq q \leq R - 2$ 且 $S[q - 1] = S[q]$ 。

对 S 建 PAM。设 v_R 为处理到位置 R 时的 $last$ ，那么 v_R 的 fail 祖先恰好对应所有以 R 结尾的回文后缀。

回文 $S[q..R]$ 的长度允许区间是 $[3, R - l]$ 。用 fail 链倍增定位长度不超过 $R - l$ 的最长祖先，再统计到长度至少为 3 为止的这一段祖先链。

solution

先把所有操作离线，得到最终大串 S 。任意时刻当前串都是 $S[l..r]$ ；一次左插入就是 $l \leftarrow l - 1$ ，一次右插入就是 $r \leftarrow r + 1$ 。

这样每次操作只会新增以新端点为端点的 CCPC 子串，旧区间内部的贡献完全不变。下面只讨论如何计算一次扩展带来的增量。

若向右扩展到 $R = r + 1$ ，新增子串一定形如 $S[p..R]$ 。由定义，它合法当且仅当 $p \leq R - 3$ ， $S[p] = S[p + 1] = S[R]$ ，且 $S[p + 1..R]$ 是回文串。

因此需要统计前缀 $S[1..R]$ 的回文后缀 $S[q..R]$ ，其中 $q = p + 1$ ，满足 $l + 1 \leq q \leq R - 2$ 且 $S[q - 1] = S[q]$ 。

对 S 建 PAM。设 v_R 为处理到位置 R 时的 $last$ ，那么 v_R 的 fail 祖先恰好对应所有以 R 结尾的回文后缀。

回文 $S[q..R]$ 的长度允许区间是 $[3, R - l]$ 。用 fail 链倍增定位长度不超过 $R - l$ 的最长祖先，再统计到长度至少为 3 为止的这一段祖先链。

链上每个结点对应一个当前出现 $S[q..R]$ ，权值为 $[S[q - 1] = S[q]]$ 。倍增表中同时维护跳过这段 fail 链时的权值和，就可以在 $\mathcal{O}(\log |S|)$ 时间内得到右扩展增量。

solution

若向左扩展到 $L = l - 1$ ，新增子串一定形如 $S[L..p]$ 。它合法当且仅当 $p \geq L + 3$ ， $S[L] = S[L + 1] = S[p]$ ，且 $S[L + 1..p]$ 是回文串。

solution

若向左扩展到 $L = l - 1$, 新增子串一定形如 $S[L..p]$ 。它合法当且仅当 $p \geq L + 3$, $S[L] = S[L + 1] = S[p]$, 且 $S[L + 1..p]$ 是回文串。

这就是在当前旧串 $S[l..r]$ 中, 统计从 l 开始、长度在 $[3, r - l + 1]$ 内的回文前缀, 并附带相邻字符相等条件 $S[L] = S[l]$ 。

solution

若向左扩展到 $L = l - 1$, 新增子串一定形如 $S[L..p]$ 。它合法当且仅当 $p \geq L + 3$, $S[L] = S[L + 1] = S[p]$, 且 $S[L + 1..p]$ 是回文串。

这就是在当前旧串 $S[l..r]$ 中, 统计从 l 开始、长度在 $[3, r - l + 1]$ 内的回文前缀, 并附带相邻字符相等条件 $S[L] = S[l]$ 。

把 S 反过来建同样的数据结构, 回文前缀查询就变成了反串上的回文后缀查询, 长度限制和链权条件完全同构。

solution

若向左扩展到 $L = l - 1$, 新增子串一定形如 $S[L..p]$ 。它合法当且仅当 $p \geq L + 3$, $S[L] = S[L + 1] = S[p]$, 且 $S[L + 1..p]$ 是回文串。

这就是在当前旧串 $S[l..r]$ 中, 统计从 l 开始、长度在 $[3, r - l + 1]$ 内的回文前缀, 并附带相邻字符相等条件 $S[L] = S[l]$ 。

把 S 反过来建同样的数据结构, 回文前缀查询就变成了反串上的回文后缀查询, 长度限制和链权条件完全同构。

因此左右两类扩展分别由正串 PAM 和反串 PAM 回答。预处理时, 对正反两棵 PAM 都保存每个位置的 $last$, $fail$ 倍增, 以及倍增段内满足相邻相等条件的权值和。

solution

若向左扩展到 $L = l - 1$, 新增子串一定形如 $S[L..p]$ 。它合法当且仅当 $p \geq L + 3$, $S[L] = S[L + 1] = S[p]$, 且 $S[L + 1..p]$ 是回文串。

这就是在当前旧串 $S[l..r]$ 中, 统计从 l 开始、长度在 $[3, r - l + 1]$ 内的回文前缀, 并附带相邻字符相等条件 $S[L] = S[l]$ 。

把 S 反过来建同样的数据结构, 回文前缀查询就变成了反串上的回文后缀查询, 长度限制和链权条件完全同构。

因此左右两类扩展分别由正串 PAM 和反串 PAM 回答。预处理时, 对正反两棵 PAM 都保存每个位置的 $last$, $fail$ 倍增, 以及倍增段内满足相邻相等条件的权值和。

离线模拟原操作, 维护当前区间 $[l, r]$ 和答案。右插入时调用正串查询, 左插入时把区间映到反串后调用反串查询。

solution

若向左扩展到 $L = l - 1$, 新增子串一定形如 $S[L..p]$ 。它合法当且仅当 $p \geq L + 3$, $S[L] = S[L + 1] = S[p]$, 且 $S[L + 1..p]$ 是回文串。

这就是在当前旧串 $S[l..r]$ 中, 统计从 l 开始、长度在 $[3, r - l + 1]$ 内的回文前缀, 并附带相邻字符相等条件 $S[L] = S[l]$ 。

把 S 反过来建同样的数据结构, 回文前缀查询就变成了反串上的回文后缀查询, 长度限制和链权条件完全同构。

因此左右两类扩展分别由正串 PAM 和反串 PAM 回答。预处理时, 对正反两棵 PAM 都保存每个位置的 *last*, *fail* 倍增, 以及倍增段内满足相邻相等条件的权值和。

离线模拟原操作, 维护当前区间 $[l, r]$ 和答案。右插入时调用正串查询, 左插入时把区间映到反串后调用反串查询。

每次扩展只新增以新端点为端点的子串, 所以把对应增量加入答案后输出即可。

solution

若向左扩展到 $L = l - 1$, 新增子串一定形如 $S[L..p]$ 。它合法当且仅当 $p \geq L + 3$, $S[L] = S[L + 1] = S[p]$, 且 $S[L + 1..p]$ 是回文串。

这就是在当前旧串 $S[l..r]$ 中, 统计从 l 开始、长度在 $[3, r - l + 1]$ 内的回文前缀, 并附带相邻字符相等条件 $S[L] = S[l]$ 。

把 S 反过来建同样的数据结构, 回文前缀查询就变成了反串上的回文后缀查询, 长度限制和链权条件完全同构。

因此左右两类扩展分别由正串 PAM 和反串 PAM 回答。预处理时, 对正反两棵 PAM 都保存每个位置的 $last$, $fail$ 倍增, 以及倍增段内满足相邻相等条件的权值和。

离线模拟原操作, 维护当前区间 $[l, r]$ 和答案。右插入时调用正串查询, 左插入时把区间映到反串后调用反串查询。

每次扩展只新增以新端点为端点的子串, 所以把对应增量加入答案后输出即可。

两棵 PAM 和倍增表的规模都是 $\mathcal{O}(|S| \log |S|)$, 单次操作 $\mathcal{O}(\log |S|)$, 总复杂度 $\mathcal{O}(|S| \log |S|)$ 。

例题

Palindrome Addicts

初始有一个空队列，支持 q 次操作： $push\ c$ 表示在队尾加入字符 c ， pop 表示弹出队首字符。

每次操作后，输出当前队列从首到尾拼成的字符串中，本质不同回文子串的数量。

$q \leq 10^6$ ， $\Sigma = \{a, \dots, z\}$ ，保证不会对空队列执行 pop 。

[题目链接](#)

solution

只看所有 *push* 形成的总串 S ，在线建立 S 的 PAM。当前队列始终是一个区间 $S[l..r]$ ，其中 r 随 *push* 增加， l 随 *pop* 增加。

solution

只看所有 *push* 形成的总串 S ，在线建立 S 的 PAM。当前队列始终是一个区间 $S[l..r]$ ，其中 r 随 *push* 增加， l 随 *pop* 增加。

对固定左端 l ，一次 *push* 后可能新增的本质不同回文串只有一个：当前前缀的最长回文后缀中，长度不超过 $r - l + 1$ 的第一个。

solution

只看所有 *push* 形成的总串 S ，在线建立 S 的 PAM。当前队列始终是一个区间 $S[l..r]$ ，其中 r 随 *push* 增加， l 随 *pop* 增加。

对固定左端 l ，一次 *push* 后可能新增的本质不同回文串只有一个：当前前缀的最长回文后缀中，长度不超过 $r - l + 1$ 的第一个。

原因是回文串的回文后缀一定是它的 border。若这个最长合法回文仍在当前区间内，那么它的所有真回文后缀也一定在区间内，不需要在本次操作中逐个加入。

solution

只看所有 *push* 形成的总串 S ，在线建立 S 的 PAM。当前队列始终是一个区间 $S[l..r]$ ，其中 r 随 *push* 增加， l 随 *pop* 增加。

对固定左端 l ，一次 *push* 后可能新增的本质不同回文串只有一个：当前前缀的最长回文后缀中，长度不超过 $r - l + 1$ 的第一个。

原因是回文串的回文后缀一定是它的 border。若这个最长合法回文仍在当前区间内，那么它的所有真回文后缀也一定在区间内，不需要在本次操作中逐个加入。

对每个 PAM 结点 u 维护 $\text{cov}(u)$ ：当前保存的一个激活出现的右端点。
 $\text{cov}(u) = 0$ 表示这个本质回文当前没有被任何标记覆盖。

只看所有 *push* 形成的总串 S ，在线建立 S 的 PAM。当前队列始终是一个区间 $S[l..r]$ ，其中 r 随 *push* 增加， l 随 *pop* 增加。

对固定左端 l ，一次 *push* 后可能新增的本质不同回文串只有一个：当前前缀的最长回文后缀中，长度不超过 $r-l+1$ 的第一个。

原因是回文串的回文后缀一定是它的 border。若这个最长合法回文仍在当前区间内，那么它的所有真回文后缀也一定在区间内，不需要在本次操作中逐个加入。

对每个 PAM 结点 u 维护 $\text{cov}(u)$: 当前保存的一个激活出现的右端点。

$\text{cov}(u) = 0$ 表示这个本质回文当前没有被任何标记覆盖。

插入字符并得到结点 u 后, 若 $\text{cov}(u) = 0$, 答案加一。随后令 $\text{cov}(u) = r$, 并把事件 (u, r) 挂到位置 $r - \text{len}(u) + 1$ 。

solution

只看所有 *push* 形成的总串 S ，在线建立 S 的 PAM。当前队列始终是一个区间 $S[l..r]$ ，其中 r 随 *push* 增加， l 随 *pop* 增加。

对固定左端 l ，一次 *push* 后可能新增的本质不同回文串只有一个：当前前缀的最长回文后缀中，长度不超过 $r - l + 1$ 的第一个。

原因是回文串的回文后缀一定是它的 border。若这个最长合法回文仍在当前区间内，那么它的所有真回文后缀也一定在区间内，不需要在本次操作中逐个加入。

对每个 PAM 结点 u 维护 $\text{cov}(u)$ ：当前保存的一个激活出现的右端点。

$\text{cov}(u) = 0$ 表示这个本质回文当前没有被任何标记覆盖。

插入字符并得到结点 u 后，若 $\text{cov}(u) = 0$ ，答案加一。随后令 $\text{cov}(u) = r$ ，并把事件 (u, r) 挂到位置 $r - \text{len}(u) + 1$ 。

朴素做法会同时更新 u 的所有 fail 祖先，因为它们也在当前区间出现。这里不这样做：只在最长的 u 上打标记，等 u 被左端删除时，再把这个右端点往 $\text{fail}(u)$ 推。

solution

为了判断一个结点是否仍被更长回文覆盖，维护 $\text{cnt}(u)$ ：当前有多少个激活标记位于 u 的 fail 儿子子树中，并直接覆盖到 u 。

solution

为了判断一个结点是否仍被更长回文覆盖，维护 $\text{cnt}(u)$ ：当前有多少个激活标记位于 u 的 fail 儿子子树中，并直接覆盖到 u 。

当第一次激活 u 时，同时令 $\text{cnt}(\text{fail}(u))$ 加一；当 u 的标记真正失效时，再把这份覆盖从父亲处删掉。

solution

为了判断一个结点是否仍被更长回文覆盖，维护 $\text{cnt}(u)$ ：当前有多少个激活标记位于 u 的 fail 儿子子树中，并直接覆盖到 u 。

当第一次激活 u 时，同时令 $\text{cnt}(\text{fail}(u))$ 加一；当 u 的标记真正失效时，再把这份覆盖从父亲处删掉。

当执行 pop 并且左端点为 l 时，处理挂在 l 上的所有事件 (u, t) 。只有在 $\text{cov}(u) = t$ 且 $\text{cnt}(u) = 0$ 时， u 记录的最后一个覆盖才真的离开当前区间。

solution

为了判断一个结点是否仍被更长回文覆盖，维护 $\text{cnt}(u)$ ：当前有多少个激活标记位于 u 的 fail 儿子子树中，并直接覆盖到 u 。

当第一次激活 u 时，同时令 $\text{cnt}(\text{fail}(u))$ 加一；当 u 的标记真正失效时，再把这份覆盖从父亲处删掉。

当执行 pop 并且左端点为 l 时，处理挂在 l 上的所有事件 (u, t) 。只有在 $\text{cov}(u) = t$ 且 $\text{cnt}(u) = 0$ 时， u 记录的最后一个覆盖才真的离开当前区间。

这时答案减一，清空 $\text{cov}(u)$ ，并把原来的右端点 t 传给 $\text{fail}(u)$ ；若父亲此前未激活，则答案重新加一。传给父亲后，同样按 $t - \text{len}(\text{fail}(u)) + 1$ 挂新的删除事件。

solution

为了判断一个结点是否仍被更长回文覆盖，维护 $\text{cnt}(u)$ ：当前有多少个激活标记位于 u 的 fail 儿子子树中，并直接覆盖到 u 。

当第一次激活 u 时，同时令 $\text{cnt}(\text{fail}(u))$ 加一；当 u 的标记真正失效时，再把这份覆盖从父亲处删掉。

当执行 pop 并且左端点为 l 时，处理挂在 l 上的所有事件 (u, t) 。只有在 $\text{cov}(u) = t$ 且 $\text{cnt}(u) = 0$ 时， u 记录的最后一个覆盖才真的离开当前区间。

这时答案减一，清空 $\text{cov}(u)$ ，并把原来的右端点 t 传给 $\text{fail}(u)$ ；若父亲此前未激活，则答案重新加一。传给父亲后，同样按 $t - \text{len}(\text{fail}(u)) + 1$ 挂新的删除事件。

一个标记每次只会沿 fail 向更短回文移动，且只有它的左端被弹出时才移动一次。所有事件总数为 $\mathcal{O}(q)$ ，因此整体复杂度为 $\mathcal{O}(q)$ 。

- 1 记号
- 2 border 相关
- 3 ACAM
- 4 回文串相关
- 5 Z 函数
- 6 后缀数据结构
- 7 基本子串结构
- 8 References

Z 函数

对字符串 s , 定义 $z_i = \text{lcp}(s, s[i..n])$ 。也就是说, z_i 是从位置 i 开始的后缀与整个串的最长公共前缀长度。

Z 函数

对字符串 s , 定义 $z_i = \text{lcp}(s, s[i..n])$ 。也就是说, z_i 是从位置 i 开始的后缀与整个串的最长公共前缀长度。

从左到右计算 z_i 。算法维护当前右端点最大的、已经确认与前缀相等的区间 $[L, R]$, 即 $s[L..R] = s[1..R - L + 1]$ 。

Z 函数

对字符串 s , 定义 $z_i = \text{lcp}(s, s[i..n])$ 。也就是说, z_i 是从位置 i 开始的后缀与整个串的最长公共前缀长度。

从左到右计算 z_i 。算法维护当前右端点最大的、已经确认与前缀相等的区间 $[L, R]$, 即 $s[L..R] = s[1..R - L + 1]$ 。

若 $i > R$, 则当前没有可用信息, 令 $z_i \leftarrow 0$ 后暴力向右扩展。

Z 函数

对字符串 s , 定义 $z_i = \text{lcp}(s, s[i..n])$ 。也就是说, z_i 是从位置 i 开始的后缀与整个串的最长公共前缀长度。

从左到右计算 z_i 。算法维护当前右端点最大的、已经确认与前缀相等的区间 $[L, R]$, 即 $s[L..R] = s[1..R - L + 1]$ 。

若 $i > R$, 则当前没有可用信息, 令 $z_i \leftarrow 0$ 后暴力向右扩展。

若 $i \leq R$, 令 $j = i - L + 1$ 。区间内部已经和前缀对齐, 所以 $z_i \geq \min(z_j, R - i + 1)$, 先用这个值初始化。

Scintilla

字符串

只有当这个初值正好顶到 R 时，才继续暴力扩展；每次成功扩展都会使 R 右移，因此总复杂度是 $O(n)$ 。

例题

The Sum of Good Numbers

称正整数为 good，当且仅当它的十进制表示中不含 0。给定由若干 good 数十进制表示串联而成的字符串 s ，以及 good 数 x 。

求两段相邻非空子串 $s[l_1..r_1]$ 与 $s[l_2..r_2]$ ，满足 $r_1 + 1 = l_2$ ，且二者作为十进制整数之和等于 x 。

$2 \leq |s| \leq 5 \times 10^5$ ， $2 \leq x < 10^{200000}$ ，且保证存在答案。

[题目链接](#)

solution

设 $m = |x|$ 。因为 x 是好数，若较大数的位数是 $m - 1$ ，那么两数位数都只能是 $m - 1$ ；否则较大数的位数就是 m 。

solution

设 $m = |x|$ 。因为 x 是好数，若较大数的位数是 $m - 1$ ，那么两数位数都只能是 $m - 1$ ；否则较大数的位数就是 m 。

第一类直接枚举：所有长度恰好为 $m - 1$ 的相邻二元组都试一遍，用 hash 判 $a + b = x$ 。

solution

设 $m = |x|$ 。因为 x 是好数，若较大数的位数是 $m - 1$ ，那么两数位数都只能是 $m - 1$ ；否则较大数的位数就是 m 。

第一类直接枚举：所有长度恰好为 $m - 1$ 的相邻二元组都试一遍，用 hash 判 $a + b = x$ 。

第二类是某个数有 m 位。用 Z 函数求出每个位置 i 和 x 的 LCP 长度 z_i ，那么短数的长度只可能是 $m - z_i - 1$ 或 $m - z_i$ 。枚举是左边还是右边，同样使用 hash 判断即可。

solution

设 $m = |x|$ 。因为 x 是好数，若较大数的位数是 $m - 1$ ，那么两数位数都只能是 $m - 1$ ；否则较大数的位数就是 m 。

第一类直接枚举：所有长度恰好为 $m - 1$ 的相邻二元组都试一遍，用 hash 判 $a + b = x$ 。

第二类是某个数有 m 位。用 Z 函数求出每个位置 i 和 x 的 LCP 长度 z_i ，那么短数的长度只可能是 $m - z_i - 1$ 或 $m - z_i$ 。枚举是左边还是右边，同样使用 hash 判断即可。

时间复杂度 $\mathcal{O}(n)$ 。

例题

Prefix of Suffixes

初始 $S = \varepsilon$ 。第 i 次操作给定 S'_i, A_i, B_i , 令
 $S_i = (S'_i + \text{lastans}) \bmod N$, 并将 S_i 追加到 S 。

设当前长度为 n , $z_i = \text{lcp}(S[i..n], S[1..n])$ 。每次操作后输出
 $\sum_{i=1}^n \sum_{j=i}^{i+z_i-1} A_j B_i$, 并令其为新的 lastans。

$N \leq 3 \times 10^5$, $0 \leq S'_i < N$, $1 \leq A_i, B_i \leq 1000$ 。

[题目链接](#)

solution

静态的 exkmp 只能一次性求出整个 z 数组。这里需要在线追加字符，因此改用 KMP 维护每个前缀的最长 border，记为 border_i 。

solution

静态的 exkmp 只能一次性求出整个 z 数组。这里需要在线追加字符，因此改用 KMP 维护每个前缀的最长 border，记为 border_i 。

对每个起点 i ， z_i 有两种状态：要么已经在某次失配时被确定下来，此后不再改变；要么当前仍然满足 $S[i..n] = S[1..n-i+1]$ ，之后每追加一个字符，它要么继续增长 1，要么在这一位失配并被确定。

solution

静态的 exkmp 只能一次性求出整个 z 数组。这里需要在线追加字符，因此改用 KMP 维护每个前缀的最长 border，记为 $border_i$ 。

对每个起点 i ， z_i 有两种状态：要么已经在某次失配时被确定下来，此后不再改变；要么当前仍然满足 $S[i..n] = S[1..n-i+1]$ ，之后每追加一个字符，它要继续增长 1，要么在这一位失配并被确定。

因而只需要维护所有还没有被确定的 z_i 。设它们的 B_i 之和为 T 。追加第 n 个字符后，删掉这一轮失配的起点，再令 $ans \leftarrow ans + A_n T$ 。

solution

静态的 exkmp 只能一次性求出整个 z 数组。这里需要在线追加字符，因此改用 KMP 维护每个前缀的最长 border，记为 border_i 。

对每个起点 i ， z_i 有两种状态：要么已经在某次失配时被确定下来，此后不再改变；要么当前仍然满足 $S[i..n] = S[1..n-i+1]$ ，之后每追加一个字符，它要么继续增长 1，要么在这一位失配并被确定。

因而只需要维护所有还没有被确定的 z_i 。设它们的 B_i 之和为 T 。追加第 n 个字符后，删掉这一轮失配的起点，再令 $\text{ans} \leftarrow \text{ans} + A_n T$ 。

这个式子只是把新出现的右端点 $j = n$ 单独拿出来：某个 i 仍未确定，当且仅当 n 落在区间 $[i, i + z_i - 1]$ 中，于是它正好贡献 $A_n B_i$ 。

solution

现在考虑如何找出这一轮失配的 i 。追加 $S[n] = c$ 前，未确定的 z_i 满足 $S[1..n-i] = S[i..n-1]$ ，也就是说 $p = n-i$ 是前缀 $S[1..n-1]$ 的 border。

solution

现在考虑如何找出这一轮失配的 i 。追加 $S[n] = c$ 前，未确定的 z_i 满足 $S[1..n-i] = S[i..n-1]$ ，也就是说 $p = n-i$ 是前缀 $S[1..n-1]$ 的 border。

把每个长度 x 向 border_x 连边，得到 KMP 的 border 树。那么所有满足上式的 p ，恰好是结点 $n-1$ 在这棵树上的祖先。

solution

现在考虑如何找出这一轮失配的 i 。追加 $S[n] = c$ 前，未确定的 z_i 满足 $S[1..n-i] = S[i..n-1]$ ，也就是说 $p = n-i$ 是前缀 $S[1..n-1]$ 的 border。

把每个长度 x 向 border_x 连边，得到 KMP 的 border 树。那么所有满足上式的 p ，恰好是结点 $n-1$ 在这棵树上的祖先。

对祖先 p ，新字符会使对应的 z_i 被确定，当且仅当下一位失配，也就是 $S[p+1] \neq c$ 。确定以后令 $i = n-p$ ，从 T 中删去 B_i 。

solution

现在考虑如何找出这一轮失配的 i 。追加 $S[n] = c$ 前，未确定的 z_i 满足 $S[1..n-i] = S[i..n-1]$ ，也就是说 $p = n-i$ 是前缀 $S[1..n-1]$ 的 border。

把每个长度 x 向 border_x 连边，得到 KMP 的 border 树。那么所有满足上式的 p ，恰好是结点 $n-1$ 在这棵树上的祖先。

对祖先 p ，新字符会使对应的 z_i 被确定，当且仅当下一位失配，也就是 $S[p+1] \neq c$ 。确定以后令 $i = n-p$ ，从 T 中删去 B_i 。

查询祖先可以按字符维护跳指针：令 $\text{fa}_{x,a}$ 表示 x 的第一个满足 $S[y+1] = a$ 的祖先 y 。于是从 $n-1$ 出发，对 $a \neq c$ 沿这些指针暴力跳，就能枚举所有本轮失配的 p 。

solution

现在考虑如何找出这一轮失配的 i 。追加 $S[n] = c$ 前，未确定的 z_i 满足 $S[1..n-i] = S[i..n-1]$ ，也就是说 $p = n-i$ 是前缀 $S[1..n-1]$ 的 border。

把每个长度 x 向 border_x 连边，得到 KMP 的 border 树。那么所有满足上式的 p ，恰好是结点 $n-1$ 在这棵树上的祖先。

对祖先 p ，新字符会使对应的 z_i 被确定，当且仅当下一位失配，也就是 $S[p+1] \neq c$ 。确定以后令 $i = n-p$ ，从 T 中删去 B_i 。

查询祖先可以按字符维护跳指针：令 $\text{fa}_{x,a}$ 表示 x 的第一个满足 $S[y+1] = a$ 的祖先 y 。于是从 $n-1$ 出发，对 $a \neq c$ 沿这些指针暴力跳，就能枚举所有本轮失配的 p 。

每个 z_i 只会被确定一次，所以这些暴力跳的总次数是线性的；跳指针用平衡树或哈希表维护即可得到 $\mathcal{O}(N \log N)$ ，精细实现可以做到 $\mathcal{O}(N)$ 。

- 1 记号
- 2 border 相关
- 3 ACAM
- 4 回文串相关
- 5 Z 函数
- 6 后缀数据结构**
- 7 基本子串结构
- 8 References

后缀数组

后缀数组 (Suffix Array, SA) 把所有后缀按字典序排列。记排序后的数组为 sa , 其逆排列为 rk 。

后缀数组

后缀数组 (Suffix Array, SA) 把所有后缀按字典序排列。记排序后的数组为 sa , 其逆排列为 rk 。

倍增构造的基本状态是: 第 k 轮已经知道每个长度 2^k 前缀块的 rank。于是长度 2^{k+1} 的块只由有序对 $(c_{k,i}, c_{k,i+2^k})$ 决定。

后缀数组

后缀数组 (Suffix Array, SA) 把所有后缀按字典序排列。记排序后的数组为 sa ，其逆排列为 rk 。

倍增构造的基本状态是：第 k 轮已经知道每个长度 2^k 前缀块的 rank。于是长度 2^{k+1} 的块只由有序对 $(c_{k,i}, c_{k,i+2^k})$ 决定。

由于两维 key 都是整数 rank，可以用两次稳定计数排序完成一轮。初始按字符排序，然后倍增长度，直到 rank 全部不同。

后缀数组

后缀数组 (Suffix Array, SA) 把所有后缀按字典序排列。记排序后的数组为 sa , 其逆排列为 rk 。

倍增构造的基本状态是: 第 k 轮已经知道每个长度 2^k 前缀块的 rank。于是长度 2^{k+1} 的块只由有序对 $(c_{k,i}, c_{k,i+2^k})$ 决定。

由于两维 key 都是整数 rank, 可以用两次稳定计数排序完成一轮。初始按字符排序, 然后倍增长度, 直到 rank 全部不同。

总复杂度 $\mathcal{O}(n \log n)$ 。

后缀数组

后缀数组 (Suffix Array, SA) 把所有后缀按字典序排列。记排序后的数组为 sa , 其逆排列为 rk 。

倍增构造的基本状态是: 第 k 轮已经知道每个长度 2^k 前缀块的 rank。于是长度 2^{k+1} 的块只由有序对 $(c_{k,i}, c_{k,i+2^k})$ 决定。

由于两维 key 都是整数 rank, 可以用两次稳定计数排序完成一轮。初始按字符排序, 然后倍增长度, 直到 rank 全部不同。

总复杂度 $\mathcal{O}(n \log n)$ 。

也存在线性的构造方法。

height 数组

定义 $\text{height}_i = \text{lcp}(s[\text{sa}_{i-1}..n], s[\text{sa}_i..n])$ 。

height 数组

定义 $\text{height}_i = \text{lcp}(s[\text{sa}_{i-1}..n], s[\text{sa}_i..n])$ 。

对任意两个后缀 x, y , 设 $a = \text{rk}_x$, $b = \text{rk}_y$, 且 $a < b$ 。那么
 $\text{lcp}(x, y) = \min_{a < i \leq b} \text{height}_i$ 。

SAM 的状态语义

SAM (Suffix Automaton) 是识别 s 的所有子串的最小自动机。它和 Trie / ACAM 最大的区别是：一个状态不对应一个确定串，而对应一整个 endpos 等价类。

SAM 的状态语义

SAM (Suffix Automaton) 是识别 s 的所有子串的最小自动机。它和 Trie / ACAM 最大的区别是：一个状态不对应一个确定串，而对应一整个 endpos 等价类。

对状态 u ，记 $\text{len}(u)$ 为该等价类中最长串的长度。这个状态表示的所有串长度恰好是 $(\text{len}(\text{link}(u)), \text{len}(u)]$ ，并且都是最长串的后缀。

SAM 的状态语义

SAM (Suffix Automaton) 是识别 s 的所有子串的最小自动机。它和 Trie / ACAM 最大的区别是：一个状态不对应一个确定串，而对应一整个 endpos 等价类。

对状态 u ，记 $\text{len}(u)$ 为该等价类中最长串的长度。这个状态表示的所有串长度恰好是 $(\text{len}(\text{link}(u)), \text{len}(u)]$ ，并且都是最长串的后缀。

suffix link 指向包含关系下的最长真后缀等价类。把 suffix link 反向后得到 parent tree；一个状态到根的路径，描述了它的最长串的所有后缀等价类。

SAM 的构造

增量构造时，加入字符 c 后新建状态 cur ，令 $\text{len}(cur) = \text{len}(last) + 1$ ，它代表所有以新位置结尾的后缀中尚未出现的部分。

SAM 的构造

增量构造时，加入字符 c 后新建状态 cur ，令 $\text{len}(cur) = \text{len}(last) + 1$ ，它代表所有以新位置结尾的后缀中尚未出现的部分。

从 $last$ 沿 suffix link 回退，把所有还没有 c 转移的状态都连向 cur 。若一直回到根都没有遇到已有转移，则 $\text{link}(cur) = root$ 。

SAM 的构造

增量构造时，加入字符 c 后新建状态 cur ，令 $\text{len}(cur) = \text{len}(last) + 1$ ，它代表所有以新位置结尾的后缀中尚未出现的部分。

从 $last$ 沿 suffix link 回退，把所有还没有 c 转移的状态都连向 cur 。若一直回到根都没有遇到已有转移，则 $\text{link}(cur) = root$ 。

若遇到 p 已经有 c 转移到 q ，并且 $\text{len}(q) = \text{len}(p) + 1$ ，则可以直接令 $\text{link}(cur) = q$ 。

SAM 的构造

增量构造时，加入字符 c 后新建状态 cur ，令 $\text{len}(cur) = \text{len}(last) + 1$ ，它代表所有以新位置结尾的后缀中尚未出现的部分。

从 $last$ 沿 suffix link 回退，把所有还没有 c 转移的状态都连向 cur 。若一直回到根都没有遇到已有转移，则 $\text{link}(cur) = root$ 。

若遇到 p 已经有 c 转移到 q ，并且 $\text{len}(q) = \text{len}(p) + 1$ ，则可以直接令 $\text{link}(cur) = q$ 。

否则要复制 q 得到 clone，令它的长度为 $\text{len}(p) + 1$ ，复制 q 的转移和 link，并把从若干祖先指向 q 的 c 转移改到 clone。最后令 q 和 cur 的 link 都指向 clone。

例题

打击复读

给定 $S \in \{A, T, G, C\}^n$, 每个位置有左权 wl_i 与右权 wr_i 。对任意子串 t , 令 $vl(t)$ 为 t 所有出现左端点的 wl 之和, $vr(t)$ 为所有出现右端点的 wr 之和, 贡献为 $vl(t)vr(t)$ 。

初始询问 $\sum_{1 \leq l \leq r \leq n} vl(S[l..r])vr(S[l..r])$, 随后 m 次单点修改 $wl_u \leftarrow v$, 每次修改后输出该值, 答案对 2^{64} 取模。

$n, m \leq 5 \times 10^5$, $0 \leq wl_i, wr_i, v < 2^{64}$ 。

[题目链接](#)

solution

先把原串翻转，修改左权变成修改右权。记翻转后的左权前缀和为 P ，对正串建 SAM；状态 u 代表长度区间 $[l_u, r_u]$ ，其中 $l_u = \text{len}(\text{link}(u)) + 1$ ， $r_u = \text{len}(u)$ 。

solution

先把原串翻转，修改左权变成修改右键。记翻转后的左权前缀和为 P ，对正串建 SAM；状态 u 代表长度区间 $[l_u, r_u]$ ，其中 $l_u = \text{len}(\text{link}(u)) + 1$ ， $r_u = \text{len}(u)$ 。

若 $i \in \text{endpos}_u$ ，则状态 u 中所有长度的左端点正好是 $i - r_u + 1, \dots, i - l_u + 1$ 。因此 u 的总贡献为 $(\sum_{i \in \text{endpos}_u} w r_i) (\sum_{i \in \text{endpos}_u} (P_{i-l_u+1} - P_{i-r_u}))$ 。

solution

先把原串翻转，修改左权变成修改右键。记翻转后的左权前缀和为 P ，对正串建 SAM；状态 u 代表长度区间 $[l_u, r_u]$ ，其中 $l_u = \text{len}(\text{link}(u)) + 1$ ， $r_u = \text{len}(u)$ 。

若 $i \in \text{endpos}_u$ ，则状态 u 中所有长度的左端点正好是 $i - r_u + 1, \dots, i - l_u + 1$ 。因此 u 的总贡献为 $(\sum_{i \in \text{endpos}_u} wr_i) (\sum_{i \in \text{endpos}_u} (P_{i-l_u+1} - P_{i-r_u}))$ 。

第二个括号只和左权有关，记为 A_u 。答案即

$\sum_u A_u \sum_{i \in \text{endpos}_u} wr_i = \sum_i wr_i \sum_{u: i \in \text{endpos}_u} A_u$ 。在 parent tree 上对每个状态子树加 A_u ，每个前缀状态查到的值就是 wr_i 的系数，所以单点修改后可以 $\mathcal{O}(1)$ 改答案。

solution

剩下只需计算 A_u 。把它拆成两部分：前半是 $\sum_{i \in \text{endpos}_u} P_{i-l_u+1}$ ，这等价于把状态 u 所代表的最短串的每次出现，映到它的左端点前缀和上。

solution

剩下只需计算 A_u 。把它拆成两部分：前半是 $\sum_{i \in \text{endpos}_u} P_{i-l_u+1}$ ，这等价于把状态 u 所代表的最短串的每次出现，映到它的左端点前缀和上。

对反串再建 SAM。正串中一个最短串出现的左端点，在反串里变成这个串反向后的右端点；用倍增在反串 SAM 中定位该最短串所属状态，就能把对应的 P 值挂到反串 parent tree 上并做子树求和。

solution

剩下只需计算 A_u 。把它拆成两部分：前半是 $\sum_{i \in \text{endpos}_u} P_{i-l_u+1}$ ，这等价于把状态 u 所代表的最短串的每次出现，映到它的左端点前缀和上。

对反串再建 SAM。正串中一个最短串出现的左端点，在反串里变成这个串反向后的右端点；用倍增在反串 SAM 中定位该最短串所属状态，就能把对应的 P 值挂到反串 parent tree 上并做子树求和。

后半 $\sum_{i \in \text{endpos}_u} P_{i-r_u}$ 可以沿 SAM 的长度区间边界转化为若干更长等价类的前半项，所以不需要另做一套结构。使用倍增定位时复杂度为 $\mathcal{O}(n \log n + m)$ ；若把定位离线下来，可以做到 $\mathcal{O}(n + m)$ 。

例题

ℳ-function

对字符串 x , 令 $z_i = \text{lcp}(x, x[i..|x|])$, 定义 $\text{zh}(x) = \sum_i z_i$ 。

给定 $s \in \Sigma^n$ 和 q 次询问 $[l, r]$, 每次输出 $\text{zh}(s[l..r])$ 。

$n, q \leq 2 \times 10^5$, $\Sigma = \{a, \dots, z\}$ 。

[题目链接](#)

solution

把串翻转。一次询问 $[l, r]$ 可以转化为 $\sum_{i=l}^r \min\{i - l + 1, \text{LCS}(i, r)\}$, 其中 $\text{LCS}(i, r)$ 是前缀 $s[1..i]$ 与 $s[1..r]$ 的最长公共后缀。这里 $i - l + 1$ 是截到询问左边界后的长度上界。

solution

把串翻转。一次询问 $[l, r]$ 可以转化为 $\sum_{i=l}^r \min\{i - l + 1, \text{LCS}(i, r)\}$, 其中 $\text{LCS}(i, r)$ 是前缀 $s[1..i]$ 与 $s[1..r]$ 的最长公共后缀。这里 $i - l + 1$ 是截到询问左边界后的长度上界。

建出 SAM。每个前缀对应一个状态 p_i , 两个前缀的 LCS 就是 p_i, p_r 在 parent tree 上 LCA 的 len。于是询问只剩下树上点对的 LCA 长度统计。

solution

把串翻转。一次询问 $[l, r]$ 可以转化为 $\sum_{i=l}^r \min\{i - l + 1, \text{LCS}(i, r)\}$, 其中 $\text{LCS}(i, r)$ 是前缀 $s[1..i]$ 与 $s[1..r]$ 的最长公共后缀。这里 $i - l + 1$ 是截到询问左边界后的长度上界。

建出 SAM。每个前缀对应一个状态 p_i , 两个前缀的 LCS 就是 p_i, p_r 在 parent tree 上 LCA 的 len。于是询问只剩下树上点对的 LCA 长度统计。

具体地说, 询问变成: 给定 parent tree 上一个点 p_r 和一段前缀点 p_i ($l \leq i \leq r$), 求 $\sum \min\{i - l + 1, \text{len}(\text{lca}(i, R))\}$ 。把询问挂到点 p_r 上, 之后用点分治统计路径经过当前分治中心的点对, 递归处理同一分支内的点对。

solution

设分治中心为 u , 只考虑 p_i, p_r 不在删去 u 后同一连通块内的贡献。若二者都在原 rooted tree 中 u 的子树内, 则它们的 LCA 就是 u , 贡献中的高度固定为 $\text{len}(u)$ 。

solution

设分治中心为 u , 只考虑 p_i, p_r 不在删去 u 后同一连通块内的贡献。若二者都在原 rooted tree 中 u 的子树内, 则它们的 LCA 就是 u , 贡献中的高度固定为 $\text{len}(u)$ 。

若 p_i 在 u 的子树内而 p_r 在外, LCA 由 p_r 到根经过 u 的那条链决定, 可在这一层预处理为固定值; 这两种情形都只需要统计若干个 $\min\{i - l + 1, h\}$ 。把 i 排序后, 用前缀和二分即可。

solution

设分治中心为 u , 只考虑 p_i, p_r 不在删去 u 后同一连通块内的贡献。若二者都在原 rooted tree 中 u 的子树内, 则它们的 LCA 就是 u , 贡献中的高度固定为 $\text{len}(u)$ 。

若 p_i 在 u 的子树内而 p_r 在外, LCA 由 p_r 到根经过 u 的那条链决定, 可在这一层预处理为固定值; 这两种情形都只需要统计若干个 $\min\{i - l + 1, h\}$ 。把 i 排序后, 用前缀和二分即可。

若 p_i 在外而 p_r 在 u 的子树内, LCA 由 p_i 决定, h 随被加入的点变化。把点按位置与 h 组织成离线二维数点, 分别维护 $h \leq i - l + 1$ 的 h 之和和其余点的 i 之和, 就能算出 $\sum \min$ 。

solution

设分治中心为 u , 只考虑 p_i, p_r 不在删去 u 后同一连通块内的贡献。若二者都在原 rooted tree 中 u 的子树内, 则它们的 LCA 就是 u , 贡献中的高度固定为 $\text{len}(u)$ 。

若 p_i 在 u 的子树内而 p_r 在外, LCA 由 p_r 到根经过 u 的那条链决定, 可在这一层预处理为固定值; 这两种情形都只需要统计若干个 $\min\{i - l + 1, h\}$ 。把 i 排序后, 用前缀和二分即可。

若 p_i 在外而 p_r 在 u 的子树内, LCA 由 p_i 决定, h 随被加入的点变化。把点按位置与 h 组织成离线二维数点, 分别维护 $h \leq i - l + 1$ 的 h 之和和其余点的 i 之和, 就能算出 $\sum \min$ 。

每个分治层只做排序、前缀和和树状数组查询, 再按分治递归扣掉同一分支内重复统计的部分。整体复杂度为 $\mathcal{O}(n \log^2 n)$ 。

- 1 记号
- 2 border 相关
- 3 ACAM
- 4 回文串相关
- 5 Z 函数
- 6 后缀数据结构
- 7 基本子串结构**
- 8 References

基本子串结构

SAM 很擅长刻画一个串的所有后缀信息。

基本子串结构

SAM 很擅长刻画一个串的所有后缀信息。

但是很多内部子串问题会同时询问左端和右端：例如一个正串 parent tree 上的结点与一个反串 parent tree 上的结点，表示的两类串到底交出哪些子串。

基本子串结构

SAM 很擅长刻画一个串的所有后缀信息。

但是很多内部子串问题会同时询问左端和右端：例如一个正串 parent tree 上的结点与一个反串 parent tree 上的结点，表示的两类串到底交出哪些子串。

基本子串结构可以看成把这两个方向放到同一张子串网格中：每个格子是一个子串，每个块是一批在左右上下文中完全同步出现的子串。

occ 与 ext

令整串为 s 。每个子串 $s[l..r]$ 对应网格中的一个格子 (l, r) ；同一列固定左端，同一行固定右端。

occ 与 ext

令整串为 s 。每个子串 $s[l..r]$ 对应网格中的一个格子 (l, r) ；同一列固定左端，同一行固定右端。

令 $\text{occ}(t)$ 表示 t 在 s 中的出现次数。对子串 t ，定义 $\text{ext}(t)$ 为最长的串 t' ，满足 t 是 t' 的子串且 $\text{occ}(t) = \text{occ}(t')$ 。

occ 与 ext

令整串为 s 。每个子串 $s[l..r]$ 对应网格中的一个格子 (l, r) ；同一列固定左端，同一行固定右端。

令 $\text{occ}(t)$ 表示 t 在 s 中的出现次数。对子串 t ，定义 $\text{ext}(t)$ 为最长的串 t' ，满足 t 是 t' 的子串且 $\text{occ}(t) = \text{occ}(t')$ 。

直观地说， $\text{ext}(t)$ 是每一次出现 t 时，前后一定会连带着一起出现的那段最大上下文。

occ 与 ext

令整串为 s 。每个子串 $s[l..r]$ 对应网格中的一个格子 (l, r) ；同一列固定左端，同一行固定右端。

令 $\text{occ}(t)$ 表示 t 在 s 中的出现次数。对子串 t ，定义 $\text{ext}(t)$ 为最长的串 t' ，满足 t 是 t' 的子串且 $\text{occ}(t) = \text{occ}(t')$ 。

直观地说， $\text{ext}(t)$ 是每一次出现 t 时，前后一定会连带着一起出现的那段最大上下文。

若 $\text{ext}(t_1) = \text{ext}(t_2)$ ，就把 t_1, t_2 放在同一个等价类中；网格中同一等价类的一个极大连通部分称为一个块。

块的形状

Lemma 6.1

每个块都是一个上端和左端对齐的阶梯形网格图。也就是说，存在 l_a, l_b, r 和一个单调不降函数 f ，使得这个块中的格子恰为

$$\{(x, y) \mid l_a \leq x \leq l_b, f(x) \leq y \leq r\}.$$

块的形状

Lemma 6.1

每个块都是一个上端和左端对齐的阶梯形网格图。也就是说，存在 l_a, l_b, r 和一个单调不降函数 f ，使得这个块中的格子恰为

$$\{(x, y) \mid l_a \leq x \leq l_b, f(x) \leq y \leq r\}.$$

称块左上角对应的串为这个等价类的代表元，记为 $\text{rep}(C)$ 。它也就是该等价类中最长的串。

块的形状

Lemma 6.1

每个块都是一个上端和左端对齐的阶梯形网格图。也就是说，存在 l_a, l_b, r 和一个单调不降函数 f ，使得这个块中的格子恰为

$$\{(x, y) \mid l_a \leq x \leq l_b, f(x) \leq y \leq r\}.$$

称块左上角对应的串为这个等价类的代表元，记为 $\text{rep}(C)$ 。它也就是该等价类中最长的串。

同一个等价类的所有块形状相同，只是在网格中平移到了不同位置；这样的平移副本数正好是 $\text{occ}(\text{rep}(C))$ 。

abbab 的基本块

$r = 5$	<i>abbab</i>	<i>bbab</i>	<i>bab</i>	<i>ab</i>	<i>b</i>
$r = 4$	<i>abba</i>	<i>bba</i>	<i>ba</i>	<i>a</i>	
$r = 3$	<i>abb</i>	<i>bb</i>	<i>b</i>		
$r = 2$	<i>ab</i>	<i>b</i>			
$r = 1$	<i>a</i>				
	$l = 1$	$l = 2$	$l = 3$	$l = 4$	$l = 5$

ababaca 的基本块

$r = 7$	<i>ababaca</i>	<i>babaca</i>	<i>abaca</i>	<i>baca</i>	<i>aca</i>	<i>ca</i>	<i>a</i>
$r = 6$	<i>ababac</i>	<i>babac</i>	<i>abac</i>	<i>bac</i>	<i>ac</i>	<i>c</i>	
$r = 5$	<i>ababa</i>	<i>baba</i>	<i>aba</i>	<i>ba</i>	<i>a</i>		
$r = 4$	<i>abab</i>	<i>bab</i>	<i>ab</i>	<i>b</i>			
$r = 3$	<i>aba</i>	<i>ba</i>	<i>a</i>				
$r = 2$	<i>ab</i>	<i>b</i>					
$r = 1$	<i>a</i>						
	$l = 1$	$l = 2$	$l = 3$	$l = 4$	$l = 5$	$l = 6$	$l = 7$

结构信息

代表元 $\text{rep}(C)$ 可以由左右两侧都不能在保持出现次数的前提下继续扩张来识别。等价地, 若 $t = s[l..r]$ 是代表元, 则向左扩到 $s[l-1..r]$ 或向右扩到 $s[l..r+1]$ 都会改变出现次数, 除非已经碰到边界。

结构信息

代表元 $\text{rep}(C)$ 可以由左右两侧都不能在保持出现次数的前提下继续扩张来识别。等价地，若 $t = s[l..r]$ 是代表元，则向左扩到 $s[l-1..r]$ 或向右扩到 $s[l..r+1]$ 都会改变出现次数，除非已经碰到边界。

块中的每一行都对应正串后缀树或 SAM 的一个状态；块中的每一列都对应反串后缀树或 SAM 的一个状态。因此块同时刻画了“共同右上下文”和“共同左上下文”。

结构信息

代表元 $\text{rep}(C)$ 可以由左右两侧都不能在保持出现次数的前提下继续扩张来识别。等价地，若 $t = s[l..r]$ 是代表元，则向左扩到 $s[l-1..r]$ 或向右扩到 $s[l..r+1]$ 都会改变出现次数，除非已经碰到边界。

块中的每一行都对应正串后缀树或 SAM 的一个状态；块中的每一列都对应反串后缀树或 SAM 的一个状态。因此块同时刻画了“共同右上下文”和“共同左上下文”。

BASS 保存两棵树、每个等价类的代表块形状，以及把任意子串定位到其等价类和块内位置的数据结构。所有代表块的周长总和是 $O(n)$ ，所以整套结构可以线性规模表示。

用于删左删右的压缩

在子串网格中，删掉左端点对应从 (l, r) 走到 $(l+1, r)$ ，删掉右端点对应从 (l, r) 走到 $(l, r-1)$ 。

用于删左删右的压缩

在子串网格中，删掉左端点对应从 (l, r) 走到 $(l+1, r)$ ，删掉右端点对应从 (l, r) 走到 $(l, r-1)$ 。

如果直接在所有 $\mathcal{O}(n^2)$ 个格子上做 DP，状态数太大；但同一个块内的子串拥有相同的 ext 和相同的出现次数，连续删左或删右在块内只是在阶梯形中平移。

用于删左删右的压缩

在子串网格中，删掉左端点对应从 (l, r) 走到 $(l+1, r)$ ，删掉右端点对应从 (l, r) 走到 $(l, r-1)$ 。

如果直接在所有 $\mathcal{O}(n^2)$ 个格子上做 DP，状态数太大；但同一个块内的子串拥有相同的 ext 和相同的出现次数，连续删左或删右在块内只是在阶梯形中平移。

因而可以把每个块压成一个结点，边只出现在相邻块之间。块内部的路径数、出口位置和 occ 的幂次由阶梯边界一次性计算。

例题

子串统计

给定字符串 S 。从 $T_0 = S$ 出发，每次删除当前串的首字符或尾字符，直到长度为 1。

对一条删除序列 $T_0, T_1, \dots, T_{n-1}$ ，定义贡献为 $\prod_{i=1}^{n-1} \text{occ}_S(T_i)$ 。
求所有删除序列贡献之和，对 998244353 取模。

$1 \leq |S| \leq 10^5$ 。

[题目链接](#)

solution

暴力跑区间 dp 是 $\mathcal{O}(n^2)$ 的。考虑对于每一个块分别进行转移。

solution

暴力跑区间 dp 是 $\mathcal{O}(n^2)$ 的。考虑对于每一个块分别进行转移。
每个块内乘上的系数都是一样的，相当于一个阶梯网格从左上边界转移到右下边界。

solution

暴力跑区间 dp 是 $\mathcal{O}(n^2)$ 的。考虑对于每一个块分别进行转移。
 每个块内乘上的系数都是一样的，相当于一个阶梯网格从左上边界转移到右下边界。
 采用经典的分治 + NTT 即可。

solution

暴力跑区间 dp 是 $\mathcal{O}(n^2)$ 的。考虑对于每一个块分别进行转移。
 每个块内乘上的系数都是一样的，相当于一个阶梯网格从左上边界转移到右下边界。
 采用经典的分治 + NTT 即可。
 时间复杂度 $\mathcal{O}(n \log^2 n)$ 。

- ① 记号
- ② border 相关
- ③ ACAM
- ④ 回文串相关
- ⑤ Z 函数
- ⑥ 后缀数据结构
- ⑦ 基本子串结构
- ⑧ References

References I

- [CDM⁺23] Jingbang Chen, Jiangqi Dai, Qiuyang Mang, Qingyu Shi, and Tingqiang Xu. Nearly Optimal Internal Dictionary Matching, 2023.
URL: <https://arxiv.org/abs/2312.11873>, arXiv:2312.11873.
- [Chen19] 陈孙立.
子串周期查询问题的相关算法及其应用.
In 国家集训队 2019 论文集. 中国计算机学会, 2019.
- [Cheng23] 程思元.
字符串 1, 2023.
APIO2023 课件.
- [com20] command_block.
Border 理论小记, 2020.
URL: <https://www.luogu.com/article/ds5cz0sg>.
- [ix3521] ix35.
NOI 一轮复习 II: 字符串, 2021.
URL: <https://www.luogu.com/article/e08ebmz6>.
- [ix3523] ix35.
字符串技术巡礼, 2023.
URL: <https://www.luogu.com/article/xz8mjesh>.
- [Jin17] 金策.
字符串算法选讲, 2017.
WC2017 课件.
- [OI 26] OI Wiki.
字符串, 2026.
URL: <https://oi-wiki.org/string/>.

References II

- [Xu23a] 徐安矣.
浅谈回文串问题的相关算法及其应用.
In 国家集训队 2023 论文集. 中国计算机学会, 2023.
- [Xu23b] 许庭强.
一类基础子串数据结构.
In 国家集训队 2023 论文集. 中国计算机学会, 2023.
- [Xu23c] 许庭强.
字符串 2, 2023.
APIO2023 课件.

致谢

谢谢大家！